

School of Electronic Engineering
and Computer Science

Final Report

Programme of study:
BSc Computer Science

Project Title:
**Personalised Audio
Deepfake Detection**

Supervisor:
Soumya Sai Vanka

Student Name:
Egor Vert

Final Year
Undergraduate Project 2025/26



Date: 12th May 2026

Acknowledgements

I would like to give a big thank you to my project supervisor, Soumya Sai Vanka, who has been very helpful, patient, and supportive throughout this year. I am very grateful for her guidance with the project and for keeping me focused and organised. I would also like to thank my amazing friends and girlfriend who have supported and stuck with me through my academic journey.

Abstract

Audio deepfakes are generated by modern text to speech systems are increasingly weaponised in voice phishing attacks that impersonate familiar voices, a problem generic detectors struggle to address because they classify audio in isolation rather than verifying speaker identity. This dissertation presents a personalised audio deepfake detection system that conditions an in domain detector on a per speaker prototype computed from a small number of reference recordings provided by the user. The detector combines a self-supervised speech encoder (WavLM) with a spectro-temporal anti-spoofing network (AASIST) under late fusion, with a Feature-wise Linear Modulation (FiLM) layer providing per speaker conditioning at inference time. On the ASVspoof 2019 LA benchmark, the fusion architecture achieves an Equal Error Rate of 1.07%, outperforming both individual streams. In a paired evaluation on the In-the-Wild benchmark, FiLM personalisation produces a statistically significant 1.30 percentage point reduction in EER (95% bootstrap CI [-2.21, -0.42], $p < 0.05$), although the effect is heterogeneous across speakers. The detector and a study web application are deployed end to end on a single self-hosted machine, and a listening study with nine informed participants establishes a human baseline of 70.6% mean accuracy on a binary detection task consisting of 20 audio clips. The work contributes an architectural baseline, an empirical characterisation of FiLM based personalisation on out of distribution audio, and a deployable infrastructure that supports continued investigation. It also documents an out of distribution score saturation finding that bounds the deployed comparison and motivates future work in calibration and broader distribution training.

C contents

Chapter 1: Introduction	7
1.1 Background	7
1.2 Problem Statement.....	8
1.3 Aim	8
1.4 Objectives.....	8
1.5 Research Questions.....	9
1.6 Contributions	9
1.7 Report Structure	10
Chapter 2: Literature Review	11
2.1 Current State of Audio Deepfakes.....	11
2.2 Traditional Detection Approaches	11
2.3 Advanced Deep Learning Architectures.....	12
2.4 Hybrid and Fusion Architectures	12
2.5 Critical Limitations and Research Gaps	13
2.6 Summary	14
Chapter 3: Design & Methodology	15
3.1 System Design Goals and Requirements	15
3.2 Detector Architecture.....	19
3.3 Personalisation: Speaker Enrolment & FiLM Conditioning.....	21
3.4 Deployable Inference Layer	23
3.5 Study Platform Architecture.....	24
3.6 User Study.....	28
Chapter 4: Implementation	30
4.1 Detector Training Pipeline	30
4.2 Personalisation: Enrolment and Prototype Storage	31
4.3 Study Web Application	31

4.4	Self-hosted Deployment	34
Chapter 5: Evaluation & Results		35
5.1	Experimental Setup	35
5.2	In Domain Benchmark – RQ1	35
5.3	Out of Distribution Evaluation – RQ2	36
5.4	Listening Study Results – RQ3	39
Chapter 6: Discussion & Conclusion		43
6.1	Limitations	43
6.2	Future Work.....	44
6.3	Overall Conclusion	45
Chapter 7: References.....		46
Chapter 8: Appendix		49
8.1	Risk Register	49
8.2	Study Ethics Approval	51
8.3	GenAI Tools – Accountability Statement.....	52

Tables

Table 1 - Software components and version	16
Table 2 - Functional requirements	17
Table 3 - Non-functional requirements	17
Table 4 - Privacy and ethics requirements	18
Table 5 - Datasets used for training and evaluation	35
Table 6 - In domain results on the ASVspoof 2019 LA dataset	36
Table 7 - In-the-Wild paired comparison results	38
Table 8 - Per session results for the nine Phase 2 participants	40
Table 9 - Risk register	50

Figures

Figure 1 - Diagram of the two stream late-fusion detector architecture	19
Figure 2 - Diagram of speaker enrolment pipeline	21
Figure 3 - Diagram of FiLM conditioning block	22
Figure 4 - Diagram showing the project deployment topology	24
Figure 5 - Study database schema	26
Figure 6 - Diagram illustrating participant flows through the study platform	28
Figure 7 - Screenshot of the Phase 1 recording page	33
Figure 8 - Screenshot of the Phase 2 listening task	34
Figure 9 - Graph showing Receiver Operating Characteristic (ROC) curve for the fusion detector across the three evaluation benchmarks	37
Figure 10 - Scatterplot showing per speaker EER on In-the-Wild	39
Figure 11 - Per participant accuracy on the 20 clip listening task	40
Figure 12 - Ethics study approval	51

Chapter 1: Introduction

1.1 Background

The cybersecurity industry is currently undergoing an extreme paradigm shift due to the rapid development and adoption of generative Artificial Intelligence (AI). With 78% of CISOs saying that AI-powered threats are having a “significant impact” on their organisations and the FBI’s 2025 report noting a 37% rise in AI-assisted business email compromise with hundreds of deepfake-based scams using cloned voices of executives and officials, it is clear that social engineering attacks weaponising generative AI have entered the mainstream (Hassan, 2025). In my professional work as a software engineer at a cybersecurity startup where we focus on protecting our clients from social engineering threats through awareness training simulations, I observe these evolving threats daily. Part of my recent work included building a vishing (voice phishing) simulation platform which would allow specific admins from our client companies to schedule and assign realistic vishing simulations for employees with the capability to clone any voice. Through this work and my experience making deepfake simulations for clients to simulate companywide attacks such as CEO fraud, I have witnessed for myself not only how sophisticated these attacks have become, but also how surprisingly simple they are to create.

Recent advancements in Text-to-Speech (TTS) and Voice Conversion (VC) technologies have made it possible to generate high-quality, hyper-realistic human speech with remarkable efficiency (Li, Chen and Wei, 2025). With modern State of the Art (SOTA) voice cloning tools such as ElevenLabs and even open source options such as Chatterbox (Resemble AI, 2025) or F5-TTS (Chen et al., 2025), it is possible to clone a person’s voice with less than 30 seconds of audio while longer samples generally provide higher quality with diminishing returns. However, while offensive capabilities are advanced and accessible, detection techniques have been falling behind. Current defensive tools often lack robustness and fail when it comes to generalisation across diverse datasets (Li, Chen and Wei, 2025), most current detection systems have great difficulty with out-of-domain examples as generalisation poses a real problem for audio deepfake detection systems (Kulkarni et al., 2024). As a result, effective audio sample verification tools remain largely inaccessible to the average consumer or fail when subjected to targeted, personalised attacks.

Drawing on my experience creating, testing, and optimising vishing simulations by getting feedback from real users and clients, one observation stood out to me: it is becoming easier to create more and more convincing audio deepfakes, especially in a vishing scenario, by adding background noise, static, and various other manipulations to the source audio to distort it before creating a voice clone. This makes the final voice clone much more believable, especially when it is further distorted by the compression degrading the audio in a regular phone call. This means that the success of detecting a well-made vishing attack, one with a believable scenario and no glaring mistakes or contradictions, depends more on context clues the target can gather consciously and subconsciously during the conversation. For instance, from my experience and observations, it is much easier to detect when a familiar person’s voice sounds different or “off” than someone who you’ve never heard before. Take for example an employee getting a sudden call from their regional manager who they might have never had a voice conversation with, just communicating over email. It would be impossible for them to know if the alleged regional manager’s voice sounds slightly off since they’ve never heard it. Conversely, if the employee got a call from their direct manager or a friend with whom they converse regularly, they would instantly be able to hear that something sounds strange and be more likely to subconsciously be on guard throughout the call.

A defensive system that addresses this kind of asymmetry needs to be personalised at inference time, taking reference audio supplied by the user and treating it as a stand-in for the listener's missing familiarity. This is the idea of outsourced familiarity, and it is the central concept this project is built around and will be referred back to regularly.

1.2 Problem Statement

Current audio deepfake detection systems focus on the binary classification of general artifacts aiming to answer the generic question of "Is this audio fake?". This approach is intuitive and works on a large scale, however it fails to address the specific personalised nature of target social engineering attacks. This report instead asks: "Is this really X person's voice?". Consequently, existing tools often lack the specificity needed to protect individuals from impersonation attacks which not only use a realistic sounding voice, but a realistic sounding voice of someone close to the target, such as a family member or friend, leveraging context and scenario building to create believable high-urgency situations which make it much more difficult to rationally assess the situation and protect oneself.

Drawing from industry experience, vishing attacks that can quickly build rapport, set up a believable story and create maximum urgency are generally the most successful. Using a voice that the target recognises can essentially bypass the rapport building stage and go straight to creating urgency. Therefore, without a way to recognise whether the voice is real or fake in real time while an attack is going on, targets are left vulnerable to making quick irrational decisions due to the perceived time pressure created by the attacker. Despite the increasing threat of voice cloning, there remains a direct absence of accessible tools that would allow a user to upload their own reference samples to create a personalised defence mechanism for themselves, their co-workers, and their families. The current literature on audio deepfake detection contains a number of architectures that perform well on benchmarks, but few architectures that condition on speaker identity at inference time, and almost none that an individual or a small organisation can practically deploy.

1.3 Aim

The primary aim of this project is to design and implement a personalised audio deepfake detection architecture that incorporates user-supplied reference audio samples at inference time. This architecture aims to address both the binary real vs fake detection problem as well as the targeted impersonation threat that motivates voice based social engineering attacks.

The secondary aim is to deploy the resulting detector as a reusable inference service and to evaluate it under realistic conditions through a study with informed user participants. The detector will be made available through both a command line interface (CLI) and an application programming interface (API) that can be used for one-off detection, or later packaged into a web app with a more accessible user interface.

1.4 Objectives

Objective 1: Design and implement a personalised audio deepfake detection architecture that combines self-supervised speech features (WavLM) with a spectro-temporal artefact detector (AASIST) under late fusion, with Feature-wise Linear Modulation (FiLM)

providing per-speaker conditioning at inference using audio samples provided by the user.

Objective 2: Train and benchmark the system against industry standard datasets to establish both in-domain performance baselines and characterise out of distribution generalisation. Evaluate on datasets such as ASVspoof 2019 LA, WaveFake, and In-the-Wild and compare against published baselines from literature.

Objective 3: Implement a speaker enrolment pipeline that builds per-speaker prototype embeddings from a small number of reference recordings and integrates them with the FiLM conditioning layer at inference time.

Objective 4: Build a deployable inference layer for the detector in the form of a FastAPI service and a command line interface.

Objective 5: Build a separate study platform in the form of a web application running locally, with a local database instance, for evaluating the system on human participants under realistic audio conditions with privacy and ethics commitments built into the system architecture.

Objective 6: Conduct a deployed listening study with informed participants to get a baseline of human performance on a binary (true/false) deepfake detection task.

1.5 Research Questions

RQ1: Does the WavLM + AASIST hybrid architecture outperform individual baselines in reducing Equal Error Rate (EER) on the ASVspoof 2019 LA benchmark?

RQ2: To what extent does FiLM based speaker conditioning, computed from a small number of reference audio recordings provided by the user, improve out of distribution detection compared to the same architecture without conditioning?

RQ3: Can a personalised audio deepfake detector be deployed and evaluated end-to-end through a study with informed participants, and what baseline accuracy do those participants achieve on a binary detection task?

1.6 Contributions

This project makes three main contributions to the area of audio deepfake detection.

The first is architectural. I design and implement a two-stream late-fusion detector that combines self-supervised speech features from WavLM with the spectro-temporal artefact features from AASIST, joined by a Multi-Layer Perceptron (MLP) head that I trained to produce a single deepfake score. On the ASVspoof 2019 LA evaluation set, this fusion model achieves an Equal Error Rate (EER) of 1.07%, outperforming both individual streams.

The second is the personalisation extension. I add a Feature-wise Linear Modulation (FiLM) layer that conditions the fused representation on a per-speaker prototype computed from a small number of user provided reference recordings at inference time. In a paired evaluation on the In-the-Wild benchmark, where the same 4,000 audio files are scored both with and without speaker conditioning, FiLM produces a statistically significant 1.30 percentage point reduction in EER (95% bootstrap CI [-2.21, -0.42], $p <$

0.05). The effect is heterogeneous across speakers and, although modest in absolute terms, it directly addresses the question of whether inference-time personalisation provides useful signal on out of distribution real world audio.

The third contribution is the deployment of the architecture and the study around it. I build a deployable inference layer that makes the detector available through both an Application Programming Interface (API) and a Command Line Interface (CLI). A local Next.js web application backed by a local Postgres database uses the API to provide a separate study platform, with privacy and ethics commitments built into the architecture itself. Using this platform, I run a deployed listening study with 9 informed participants which establishes a human baseline of 70.6% mean accuracy on a binary detection task consisting of 20 audio clips, and surfaces a calibration limitation in the deployed detector that motivates future work discussed in Chapter 6.

1.7 Report Structure

Chapter 1: Introduction, overview of the problem, project aims, objectives, contributions, and research questions

Chapter 2: Literature review, analysis of audio deepfake generation tools, strengths and limitations of current detection models, struggles with generalisation and the personalisation gap

Chapter 3: System design and methodology, covering the architecture of WavLM + AASIST + FiLM detector, speaker enrolment pipeline, the deployed inference layer, and the design of the study platform used to evaluate the system on human participants

Chapter 4: Implementation, covering the detector training pipeline, the inference service and command line interface, the study web application and data pipeline, and the production self-hosted deployment

Chapter 5: Evaluation and results, covering in-domain benchmarks on ASVspoof 2019 LA, out of distribution evaluation on WaveFake and In-the-Wild including the paired FiLM personalisation comparison, and the deployed listening study with informed participants

Chapter 6: Discussion and conclusion, including a critical analysis of the results, limitations of the work, and directions for further research

Chapter 7: References

Chapter 8: Appendices

Chapter 2: Literature Review

2.1 Current State of Audio Deepfakes

The rapid evolution of generative AI has completely changed the current landscape of social engineering attacks, especially audio and video deepfakes. Services such as ElevenLabs (2025) allow users to create hyper-realistic clones of human voices from as little as 10 seconds of audio. Moreover, open source options have recently emerged such as Sesame (Iribe, Kumar and Sesame Team, 2025), F5-TTS (Chen et al., 2025), and Chatterbox (Resemble AI, 2025) which also offer state of the art (SOTA) voice cloning and text to speech (TTS) functionality on par with closed source options, allowing anyone with technical knowhow to run these models locally without restrictions. All of the aforementioned tools have the capability to generate human expressive speech which mimics emotional intonations, making it even more difficult to recognise the spoofed audio. With the line between AI-generated audio and real human speech becoming blurrier, the danger of misinformation and fabricated narratives increases as high profile public figures can have their voices cloned to make potentially harmful deepfakes. As generative models continue to improve, detection models are struggling to keep pace, creating a significant defensive lag (Li, Chen and Wei, 2025).

2.2 Traditional Detection Approaches

Historically, spoofed audio detection relied heavily on hand-crafted features to tell apart genuine speech from synthetic speech. A standard approach involves the use of Mel-Frequency Cepstral Coefficients (MFCCs) as they can model the audio signal's short-term power spectrum. The perceptible difference between lower audio frequencies is much more significant than higher frequencies (Loo, 2020) so, by emphasising lower frequency components where there is more speech information, MFCCs can represent the most crucial spectral characteristics of an audio sample, making them a good choice for synthetic speech detection (Asuai et al., 2025). Research by Priya et al. (2025) showed that extracting MFCCs and using a Random Forest classifier can effectively identify deepfake content by finding subtle inconsistencies in the audio signals. Similarly, Asuai et al. (2025) note that while real audio exhibits smooth spectral transitions, synthetic audio often displays unnatural high-frequency artifacts visible in spectrogram analysis.

Despite their utility, traditional feature engineering methods face significant limitations, particularly when evaluated across multiple synthesis architectures or on out of distribution data. Models that rely solely on specific feature sets, such as MFCCs, often hit a performance ceiling as they may fail to capture subtle phase-related artifacts introduced by advanced neural vocoders which suggests the need for complementary feature types (Asuai et al., 2025). Moreover, traditional machine learning models such as XGBoost have proven to be less effective at identifying fake audio, achieving lower accuracy scores when compared to Convolutional Neural Networks (CNNs) and Bidirectional Long Short-Term Memory (BiLSTM) networks (Thawani et al., 2025). The reliance on manual feature extraction can also limit scalability and real-time detection capabilities (Asuai et al., 2025).

2.3 Advanced Deep Learning Architectures

2.3.1 Self-Supervised Learning (SSL) Models: WavLM

To overcome the limitations of handcrafted feature extraction, recent research has moved towards SSL models that learn representations directly from raw audio. WavLM is a prominent example, designed to solve full-stack speech processing tasks by jointly learning masked speech prediction and denoising during the pre-training stage (Chen et al., 2022). Architecturally, WavLM extends earlier SSL frameworks such as Wav2Vec2 and HuBERT with the addition of a denoising objective during pre-training and gated relative position bias in its transformer layers, both of which were introduced to improve robustness on noisy and overlapping speech (Chen et al., 2022). Utilising a training dataset of 94k hours of diverse audio, WavLM learns universal representations that improve performance on non-ASR tasks, including speaker verification and deepfake detection (Chen et al., 2022). In comparative evaluations, WavLM-based systems consistently outperformed other SSL models such as Wav2Vec2 and HuBERT in deepfake detection scenarios which demonstrates superior efficiency and effectiveness as a frontend feature extractor (Kulkarni et al., 2024).

2.3.2 Graph-Based Analysis: AASIST

Alongside SSL advancements, graph-based architectures emerged to model the spectro-temporal relationships in audio. The Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks (AASIST) architecture processes raw waveforms through a sinc-convolutional front end and applies graph neural networks to capture complex dependencies (Jung et al., 2021). AASIST utilises a heterogeneous stacking graph attention layer and a max graph operation to highlight the most relevant features, creating a unified spectro-temporal graph which contains all the information needed to make a final decision (Luo and Sivasundari, n.d.). Despite being effective as a standalone system, with the original paper reporting a pooled EER of 1.13% (0.83% best seed) on the ASVspoof 2019 LA evaluation set (Jung et al., 2021), AASIST has shown moderate performance when evaluated against diverse, unseen datasets, prompting investigation into its use as a backend classifier within larger frameworks (Kulkarni et al., 2024).

2.3.3 Conditioning Mechanisms: FiLM

Beyond raw feature extraction and classification, recent work has explored mechanisms for conditioning a network's behaviour on auxiliary input at inference time, without retraining the underlying weights. Feature-wise Linear Modulation (FiLM), introduced by Perez et al. (2017) for visual reasoning, is one such mechanism that applies learned scale and shift parameters to a network's intermediate features conditioned on an external signal, allowing the same model to behave differently depending on the context. FiLM has since been used as a general conditioning approach across domains including image stylisation and speech recognition. Its application to audio deepfake detection, particularly conditioned on user supplied reference audio, remains largely unexplored.

2.4 Hybrid and Fusion Architectures

2.4.1 Rationale for Combining Complementary Feature Signals

Based on current literature, it is difficult to identify a single model that consistently outperforms all others across all attack types, making the necessity for hybrid architectures that can combine complementary strengths clear. For instance, the integration of spatial features extracted by CNNs and temporal dependencies captured by Long Short-Term Memory (LSTM) networks has been shown to enhance detection

accuracy (Asuai et al., 2025). The rationale is that fusing different feature representations, such as spectral features from SSL models and spectro-temporal graphs from AASIST, can capture a more holistic set of deepfake artifacts that individual models might miss (Asuai et al., 2025; Luo and Sivasundari, n.d.).

2.4.2 Review of Existing Hybrid Architectures

Several hybrid architecture implementations have been explored to test this rationale. Thawani et al. (2025) proposed a stacking model integrating RNN, CNN, BiLSTM, GRU, and XGBoost, achieving 94.03% accuracy on the Fake-or-Real (FoR) dataset using ensemble learning. Another noteworthy approach is the Whisper + AASIST architecture, where the OpenAI Whisper model was used for frontend feature extraction feeding into an AASIST backend (Luo and Sivasundari, n.d.). Luo and Sivasundari (n.d.) note that their proposed method failed to surpass their initial expectations, noting hardware limitations and workarounds such as fine-tuning using less memory-intensive Whisper models. However, their experiment proved that such workarounds could still yield competitive Equal Error Rates (EER) on standard benchmarks.

2.4.3 Effectiveness of fusion heads in lowering EER

Empirical evidence supports the efficacy of fusion strategies in lowering error rates. Kulkarni et al. (2024) demonstrated that an ensemble fusion system (S10) combining scores from WavLM, HuBERT, and other models achieved a minimum Detection Cost Function (minDCF) of 0.164 and the lowest EER of 6.41% on the ASVspoof 2024 progress set. Similarly, Asuai et al. (2025) reported that a hybrid CNN-LSTM model achieved an accuracy of 94.7% on the Fake-or-Real (FoR) dataset, surpassing standalone model performance with CNN achieving 87.3% accuracy and LSTM achieving 82.7% accuracy. These results indicate that fusion heads can effectively combine the complementary strengths of each model, thereby reducing the chance of detection errors and improving robustness across different types of audio manipulations. While fusion architectures have also been shown to improve out of distribution performance over single model baselines, the magnitude of these gains varies considerably across fusion designs and evaluation conditions, and significant generalisation gaps often remain even with the best ensemble systems (Kulkarni et al., 2024).

2.5 Critical Limitations and Research Gaps

2.5.1 The Generalisation Problem

Despite these advancements, the critical limitation that is faced by almost all current models is their inability to generalise. Evaluation frameworks such as SONAR reveal that while foundation models perform better than traditional methods, their performance still suffers significantly when tested against advanced TTS tools like OpenAI’s Voice Engine (Li, Chen and Wei, 2025). Kulkarni et al. (2024) corroborate this, observing that SOTA systems show poor generalisation on the ASVspoof 2024 dataset compared to 2019, highlighting the challenge of bias and variance in supervised learning for out-of-domain scenarios. Real-world generalisation has been further studied through purpose-built benchmarks such as the In-the-Wild dataset (Müller et al., 2026), which collects bonafide and deepfake audio of public figures from publicly available sources including social networks and video sharing platforms to evaluate detector robustness on realistic audio samples rather than clean studio recordings.

The complexity of target speech, including variations in the codec, channel, and noise, often leads to detection models incorrectly categorising audio samples. Xie et al. (2023) argue that existing methods rely too heavily on deep backbone networks that overfit to

the source domain. To address this, Xie et al. (2023) proposes Domain-Invariant Representation Learning (DIRL) strategies, drawing inspiration on their successful application in video anti-spoofing detection systems and applying them to audio by focusing on differences caused by factors such as environmental noise, recording equipment, and deception attack methods.

2.5.2 The Personalisation Gap

There is a distinct gap in the current literature which is the lack of personalised detection systems. Most current solutions focus on a binary classification of real versus fake audio without verifying speaker identity (Li, Chen and Wei, 2025). However, with the rise of AI-driven business email compromise (BEC) and executive impersonation involving cloned voices (Hassan, 2025), there is a growing need for systems that validate whether a voice truly belongs to a specific individual. Li, Chen and Wei (2025) found that few-shot fine-tuning can significantly improve detection on specific datasets, suggesting a potential pathway for personalised, identity-aware protection.

2.6 Summary

The literature establishes that while SSL models like WavLM and graph-based architectures like AASIST offer robust baselines, they struggle individually with generalisation and unseen attacks, which is a growing issue considering the rapid rate at which new SOTA TTS models are released with constantly improving voice cloning capabilities. Hybrid architectures and ensemble methods have proven effective in lowering EER, yet they often fail to address the specific threat of targeted identity impersonation. As a result, there is a clear justification for a novel approach that combines the feature extraction power of WavLM with the structural analysis of AASIST, augmented by FiLM-based speaker conditioning at inference time to address the personalisation gap and improve robustness against real-world threats.

Chapter 3: Design & Methodology

3.1 System Design Goals and Requirements

This section sets out the functional and non-functional requirements that shape the rest of the chapter. The requirements are derived directly from the project objectives in Section 1.4, the personalisation gap identified in Section 2.5.2, and the ethics constraints. Each requirement is given a unique identifier (F for functional, NF for non-functional, P for privacy and ethics) so that it can be traced through the implementation in Chapter 4 and Chapter 5.

3.1.1 Hardware Requirements

The following hardware is the minimum recommended specification for deploying and using the system. Specific configurations used during development are noted where relevant.

3.1.1.1 Desktop or Laptop with Sufficient CPU and Memory

Required for running the inference service and supporting development. A modern multicore CPU and at least 16GB of RAM are recommended for smooth detector inference. Development was carried out on an Apple M1 Pro with 16GB of unified memory, which used Metal Performance Shaders (MPS) acceleration. The equivalent x86 hardware would also be sufficient, preferably with CUDA acceleration for systems with compatible Nvidia GPUs.

3.1.1.2 Always-on Deployment Machine with a Stable Internet Connection

Required to host the study platform and inference service for the duration of the study. Any reliable always-on Mac, Linux or Windows machine with a stable internet connection is suitable. Alternatively, a hosted VPS can also be used. The study platform is exposed publicly via Tailscale Funnel, which avoids the need for a static IP or port-forwarding configuration.

3.1.1.3 Microphone

Required for participants to record voice samples during Phase 1 of the study. A standard built-in laptop or smartphone microphone is sufficient.

3.1.1.4 Audio Output Device (Speakers or Headphones)

Required for participants to listen to clips during Phase 2 of the study. Headphones are recommended for cleaner listening conditions and to reduce background noise that may interfere with proper hearing of the audio samples.

3.1.2 Software Requirements

The components below were used in the implementation. Versions reflect those at the time of evaluation.

Component	Version	Purpose
Python	3.11	Detector training, inference service, CLI
PyTorch	2.x	Deep learning framework with MPS backend

Hugging Face Transformers	4.x	WavLM checkpoint loading
FastAPI	0.111	Inference service over HTTP
Next.js	16	Study web application front end
Supabase (local)	2024.x	Postgres database and object storage
Caddy	2.x	Reverse proxy with automatic HTTPS
PM2	5.x	Process supervision for the three deployed apps
Tailscale	1.x	Public HTTPS via Funnel
F5-TTS	1.0	Deepfake clip generation for Phase 2 of study

Table 1 - Software components and version

3.1.3 Functional Requirements

ID	Requirement	Source	Status
F1	The system is able to classify a single audio clip as bonafide or synthetic, returning a single deepfake score	Obj 1, RQ 1	✓
F2	The fused detector will combine WavLM and AASIST features through a Multi-Layer Perceptron head trained from scratch	Obj 1, RQ 1	✓
F3	The system provides an enrolment interface (HTTP endpoint and CLI command) that accepts 3-5 bonafide reference recordings per speaker, computes a 1696-d L2-normalised prototype, and persists it to a local prototype database for retrieval at inference time	Obj 3	✓
F4	A FiLM layer will apply learned per-speaker scale and shift parameters to the fused representation when a prototype is supplied	Obj 1, RQ 2	✓
F5	When no prototype is supplied, the system automatically falls back to the unconditioned detector with no performance penalty	Obj 1, RQ 2	✓
F6	The detector is accessible through both an HTTP API and a command line interface	Obj 4	✓

F7	The system supports common audio input formats including WAV, MP3, and FLAC	Obj 4	✓
F8	The detector supports batch processing of multiple audio files in a single invocation, used during evaluation runs on benchmark datasets	Obj 2, Obj 4	✓
F9	All detection results from both the API and CLI are returned as JSON which can be parsed or read directly	Obj 4	✓
F10	The study platform will guide informed participants through Phase 1 (record 5 prompted sentences) and Phase 2 (20 clip listening test with real/fake response and 1-5 confidence scoring) and record data anonymously for analysis	Obj 5, Obj 6	✓

Table 2 - Functional requirements

3.1.4 Non-Functional Requirements

ID	Requirement	Target	Status
NF1	In domain equal error rate on ASVspoof 2019 LA evaluation set	$\leq 2.0\%$	✓
NF2	Inference latency for a single 4 second long clip on standard consumer hardware	< 2 seconds	✓
NF3	Fully local data processing during participant interaction in the study	No third party network calls	✓
NF4	Browser support for the study platform including mobile responsiveness	Latest Chrome, Firefox, Safari	✓
NF5	Detector robustness against unseen synthesis architectures measured and reported	Reported on WaveFake and In-the-Wild benchmarks as a limitation in Chapter 5	partial
NF6	Easy to switch between detector variants for evaluation (WavLM/AASIST only, fusion, fusion + FiLM)	Selectable via a CLI flag or an API config parameter at startup without requiring any code changes	✓
NF7	Minimise the number of reference samples needed for speaker enrolment	3-5 samples per speaker	✓

Table 3 - Non-functional requirements

3.1.5 Privacy and Ethics Requirements

The system is designed to handle participant audio and personal data responsibly. The following commitments shape the architecture itself rather than being added as an afterthought.

ID	Commitment	Enforcement mechanism	Status
P1	All audio processing performed locally	F5-TTS and detector hosted on the deployment machine (a local mac-mini), exposed only via Tailscale Funnel	✓
P2	Participant email stored separately from session data	phase1_emails table has no foreign key and no timestamp	✓
P3	Anonymous UUID identifier per participant	participants.id is a UUID generated automatically, no real names are stored	✓
P4	No personally identifiable information in any logs	vdetect/logging.py and webapp/lib/logging.ts scrub UUIDs and emails	✓
P5	Data accessible only to the research team	Private Supabase storage buckets, service-role API, all data stored on local mac-mini	✓
P6	Withdrawal possible until submission	Phase 1 batch upload with rollback if the closing page is left before pressing the Finish button	✓
P7	No third-party network calls during participant session	Telemetry disabled in ecosystem.config.js, no analytics SDKs included	✓
P8	Synthetic clips constrained to non-offensive content	generate_deepfakes.py synthesises only the five sentences that have been approved by the QMUL ethics team	✓

Table 4 - Privacy and ethics requirements

3.2 Detector Architecture

The detector is a two-stream late-fusion model that combines a self-supervised speech encoder (WavLM) with a purpose built spectro-temporal anti-spoofing network (AASIST). The motivation for combining these two representations was set out in Section 2.4.1: neither stream alone captures the full set of cues a deepfake might leave behind. WavLM provides a general phonetic and prosodic representation learned from 94k hours of diverse speech, while AASIST provides spoofing-specific spectro-temporal features learned directly from synthesis artefacts. The full forward pass is illustrated in Figure 3.1

WavLM and AASIST features fused into a 1696-d representation, optionally modulated by a per-speaker FiLM layer.

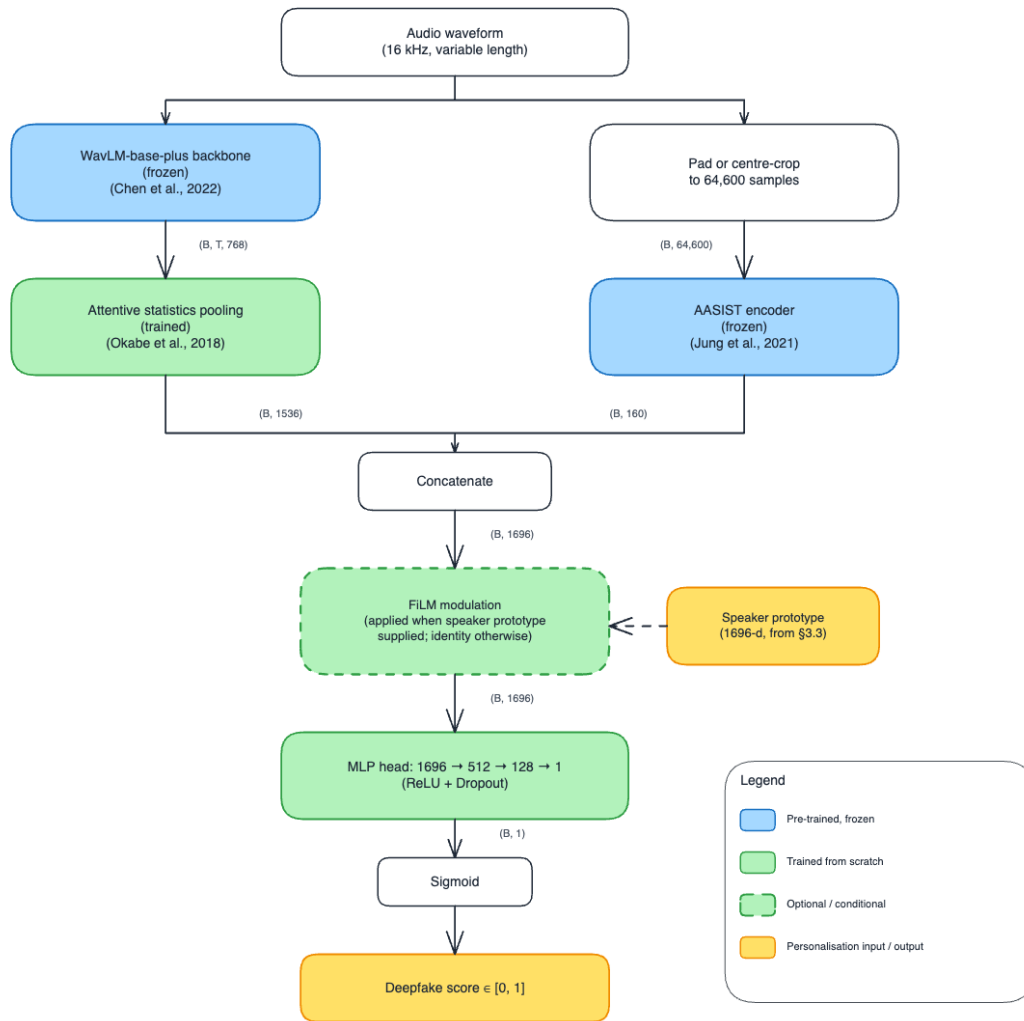


Figure 1 - Diagram of the two stream late-fusion detector architecture

3.2.1 WavLM Stream

WavLM (Chen et al., 2022) is a pre-trained transformer model that processes a raw waveform in short overlapping slices, called frames, and produces one 768-dimensional vector per frame at roughly 50 frames per second. For a four second clip, this means roughly 200 frame-level vectors, each of which captures information about the small slice of audio it represents.

The classification head, however, only needs a single vector per clip, so the frame sequence must be summarised into one. Simply averaging all the frames would treat silence and speech equally, which discards useful structure. Instead, the WavLM stream uses attentive statistics pooling (Okabe, Koshinaka and Shinoda, 2018), which learns to summarise the frames in a more informative way. A small attention network (a $768 \rightarrow 128 \rightarrow 1$ Multi-Layer Perceptron with a Tanh non-linearity) reads each frame and outputs a single score per frame. These scores are then softmax normalised across the time axis so they sum to one. The resulting weights are used to compute an attention-weighted mean and an attention-weighted standard deviation across the frames, which are concatenated to form a single 1536-dimensional utterance level vector. The mean captures what the speaker sounds like on average while the standard deviation captures how much that signal varies across the clip.

During training, the WavLM backbone itself is held frozen so that only the small attention network inside the pooling layer is updated. Freezing the backbone reflects the consumer hardware constraint identified in Section 3.1, and avoids overwriting the self-supervised representation that gives WavLM its generalisation properties.

3.2.2 AASIST Stream

In parallel, the same waveform is fed to the official AASIST encoder (Jung et al., 2021), using weights pretrained by the original authors on ASVspoof 2019 LA. AASIST expects a fixed input length of 64,600 samples, which is roughly four seconds at 16 kHz, so each waveform is centre cropped or zero padded to this length before the stream begins.

Internally, AASIST first applies a sinc-convolutional front end, which is a learnable filter bank inspired by how the human ear separates audio into frequency bands. This produces a time-frequency representation of the clip, conceptually similar to a spectrogram, but with the filter shapes learned from the data rather than fixed in advance. This representation is then passed through four stages of two-dimensional convolution and pooling, which progressively compress and abstract the features. Finally, a heterogeneous stacking graph attention layer treats the temporal and spectral features as two interacting graphs and produces a compact embedding. This is formed by concatenating five 32-dimensional pooled features (max pooled and average pooled temporal features, max pooled and average pooled spectral features, and a learned master node), giving a 160-dimensional output vector.

As with WavLM, this stream is held frozen, so the fusion head learns to combine pre-trained features rather than re-learning them.

3.2.3 Late Fusion and Classification Head

The 1536-dimensional WavLM embedding and the 160-dimensional AASIST embedding are concatenated end to end, producing a single 1696-dimensional vector that contains both views of the audio. This vector is optionally modulated by the FiLM layer described in Section 3.3, which applies a per-speaker scaling and shifting when a speaker prototype is supplied. When no prototype is given, the FiLM layer is bypassed and the fused vector passes through to the classification head unchanged.

The result is then passed through a small MLP classification head with two hidden layers of size 512 and 128, both followed by a ReLU non-linearity and dropout (rates of 0.3 and 0.2 respectively) to reduce overfitting. The final layer outputs a single number, which is passed through a sigmoid function to produce a deepfake score in the $[0, 1]$ range, where values closer to 0 indicate bonafide audio and values closer to 1 indicate synthetic audio.

Together, the fusion head, the WavLM pooling network, and the FiLM layer account for around 2.8 million trainable parameters. The two pre-trained backbones, which are held frozen throughout training, account for the remaining 94 million or so.

3.3 Personalisation: Speaker Enrolment & FiLM Conditioning

Section 3.2 described the unconditioned detector, which gives the same score to a given clip regardless of who the speaker is. This section describes how that detector is actually personalised to a known speaker using a small amount of bonafide audio from a target speaker which is reduced to a compact “prototype” vector, which is in turn used at inference time to modulate the detector’s activations.

3.3.1 Motivation

Section 2.5.2 identified the generalisation gap as a central weakness of modern deepfake detection systems. A model that achieves around a 1% EER on the dataset it was trained on can degrade by more than an order of magnitude on out of distribution audio. Among the factors contributing to this gap, one stands out as actionable in a deployed setting: the detector knows nothing about the specific person whose voice it is supposed to be protecting. If we can get even a small amount of bonafide audio for a specific person, the detector can in principle adapt its decision boundary to better separate their genuine voice from any synthetic imitation. This is what the personalisation pipeline in this project is designed to do.

3.3.2 Speaker Enrolment and Prototype Computation

To make personalisation tractable, each enrolled speaker is reduced to a single fixed-size vector called a prototype. The enrolment interface specified in F3 of Section 3.1 accepts 3-5 bonafide reference recordings for a given speaker. Each recording is passed through the WavLM and AASIST streams described in Section 3.2 and the resulting fused 1696-dimensional embeddings are stacked together. These are then averaged element wise across the recordings, which produces a single vector that captures what the speaker tends to sound like across multiple utterances rather than what they happened to say in any one recording. The average is L2-normalised so that all prototypes have unit length, which means that the conditioning signal depends only on the direction of the vector and not on its magnitude. The resulting prototype is then stored in a local JSON database, keyed by the speaker’s anonymous identifier, and looked up whenever that speaker requests a personalised detection.

3-5 reference recordings → fused embeddings → mean → L2 normalise → speaker prototype → database.

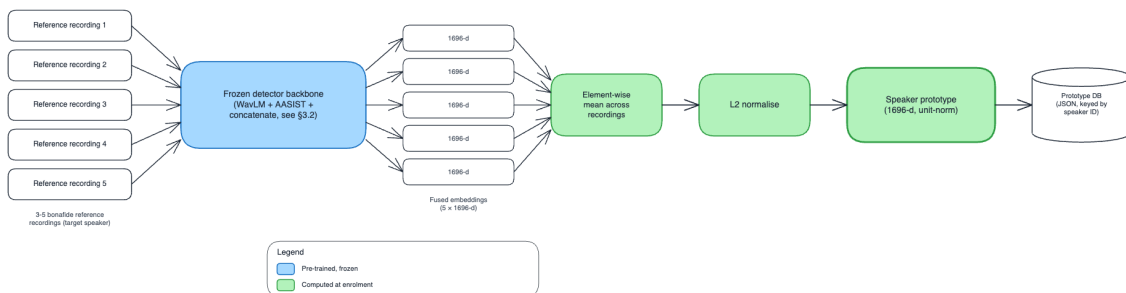


Figure 2 - Diagram of speaker enrolment pipeline

3.3.3 FiLM Conditioning

Feature-wise Linear Modulation, or FiLM, is a lightweight conditioning mechanism originally introduced by Perez et al. (2017) for visual question answering. It allows a side input, in this case the speaker prototype, to influence the activations of a main network without changing its overall structure. A FiLM block computes:

$$\text{Output} = (1 + \chi(c)) \odot \text{features} + \beta(c)$$

Where *features* is the 1696-dimensional fused embedding produced by the two streams, *c* is the 1696-dimensional speaker prototype, and $\odot$ denotes element-wise multiplication. The two functions $\chi(c)$ and $\beta(c)$ are themselves small two-layer MLPs (each a Linear(1696→256) followed by a ReLU and a Linear(256→1696)) that take the prototype as input and produce per-dimension scaling and shifting vectors. Together, they allow the network to amplify certain dimensions of the fused embedding and dampen others based on who the speaker is.

FiLM was chosen over alternatives such as concatenation or cross-attention for three reasons. Firstly, it is parameter efficient – the two MLPs together account for roughly 1.7 million trainable parameters, an order of magnitude smaller than a comparable attention-based conditioning layer over the same dimensionality. Secondly, FiLM operates on the fused embedding after the two backbones, which means both pre-trained encoders remain untouched and the personalisation signal does not need to be back propagated through them. Finally, FiLM admits a particularly clean initialisation that allows training to begin from the unconditioned baseline, as described next.

Per-speaker γ and β produced by two small MLPs modulate the 1696-d fused embedding. Init-to-identity at training start.

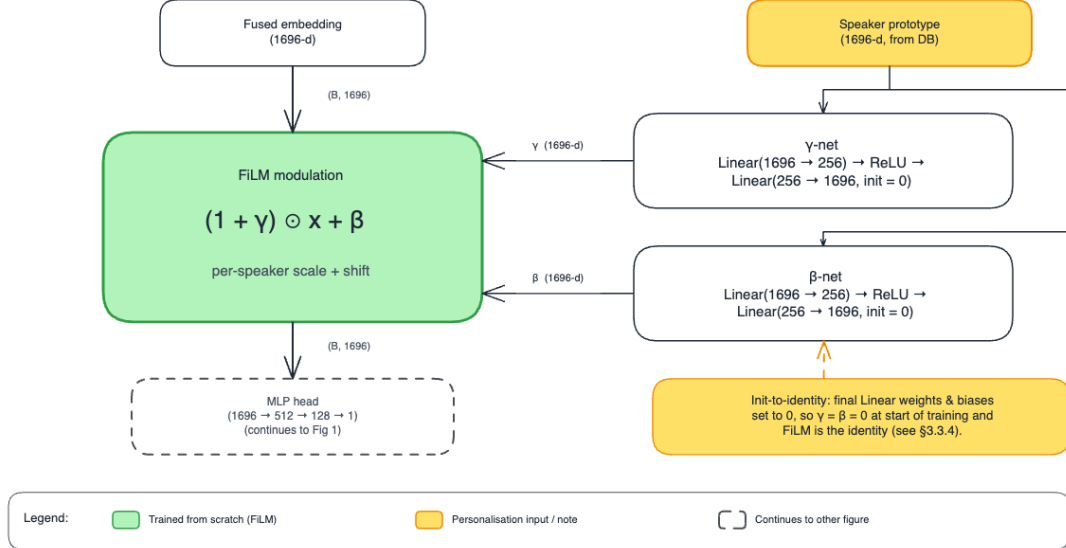


Figure 3 - Diagram of FiLM conditioning block

3.3.4 Init-to-identity Initialisation

The final layer of both $\chi(c)$ and $\beta(c)$ is initialised with all weights and biases set to zero. This means that at the very start of training, $\chi(c) = 0$ and $\beta(c) = 0$ for any speaker prototype, and the FiLM block computes $(1 + 0) \odot \text{features} + 0 = \text{features}$. In other words, FiLM starts as the identity function and the personalised detector starts behaving exactly like the unconditioned detector. As training proceeds, the gradient signal from the classification loss can move γ and β away from zero only where doing so improves the loss. If personalisation turns out not to help for a particular speaker, the FiLM block

can stay close to identity and the detector behaves no worse than its unconditioned counterpart. Personalisation in this design is intended to augment the underlying detector rather than replace it. The init-to-identity property guarantees this at the start of training, however whether it continues to hold for unseen speakers at test time is an empirical question, taken up in Chapter 5.

3.4 Deployable Inference Layer

The detector described in Sections 3.2 and 3.3 is, by itself, a PyTorch object. While usable for local offline experiments, it is not directly usable by the study platform or by any application that wants to query the detector over a network. This section describes the thin deployable layer that wraps the detector, the dual command line and HTTP entry points it exposes, and the local hosting setup used to serve participants during the listening study without exposing data to any third parties.

3.4.1 Shared Service Layer

Both the command line tool and the HTTP service wrap a single Python class – `DetectionService`, which is responsible for loading the model checkpoint, enrolling speakers, and running inference. Sharing this implementation means that a bug fix or a model update applies uniformly to both entry points, and that the two interfaces cannot drift apart behaviourally. The command line tool is intended for local, offline use, primarily running benchmark evaluations and producing per-speaker analyses reported in Chapter 5. The HTTP service is intended for online use, primarily by the study web application and by any future applications that need to send requests to the detector.

```
class DetectionService:
    # Shared inference + enrolment logic used by the CLI and the FastAPI
    server

    def __init__(self) -> None:
        self._model: Optional[torch.nn.Module] = None
        self._model_type: Optional[ModelType] = None
        self._device: str = ""
        self._max_len: int = 64000
        self._lock = threading.Lock()
```

3.4.2 HTTP Service

The HTTP service is implemented using FastAPI and exposes five endpoints: `/detect` for a single audio clip, `/batch-detect` for multiple audio clips in a single request, `/enroll` for speaker enrolment from 3-5 reference recordings, `/info` for inspecting checkpoint metadata, and `/health` for checking that the service is alive. The model checkpoint is loaded once at process start via FastAPI’s lifespan event handler and held in memory across requests, so the per-request latency reflects only the forward pass and not the 5-10 seconds that loading the fusion checkpoint can take. All responses are returned in JSON following the functional requirement F9.

3.4.3 Deployment topology

The full deployment runs on a single Apple Mac mini and consists of three processes supervised by the PM2 process manager: the FastAPI inference service, the Next.js study web app described in Section 3.5, and a background worker that drains an asynchronous enrolment job queue so that long running fusion forward passes are kept off the request path. A Caddy reverse proxy sits in front of all three services, routing requests matching `/api/vdetect/*` to the inference service and everything else to the web application. Public HTTPS is provided by Tailscale Funnel, which terminates TLS

upstream of the Mac mini and forward plain HTTP to Caddy on a private interface. This arrangement avoids exposing the Mac mini's residential IP address, removes the need for port forwarding or a dedicated server, and keeps every participant interaction within the local network which is a direct enforcement of privacy requirement P1.

Public HTTPS via Tailscale Funnel. Caddy routes `/api/vdetect/*` to the FastAPI service and everything else to the Next.js webapp.

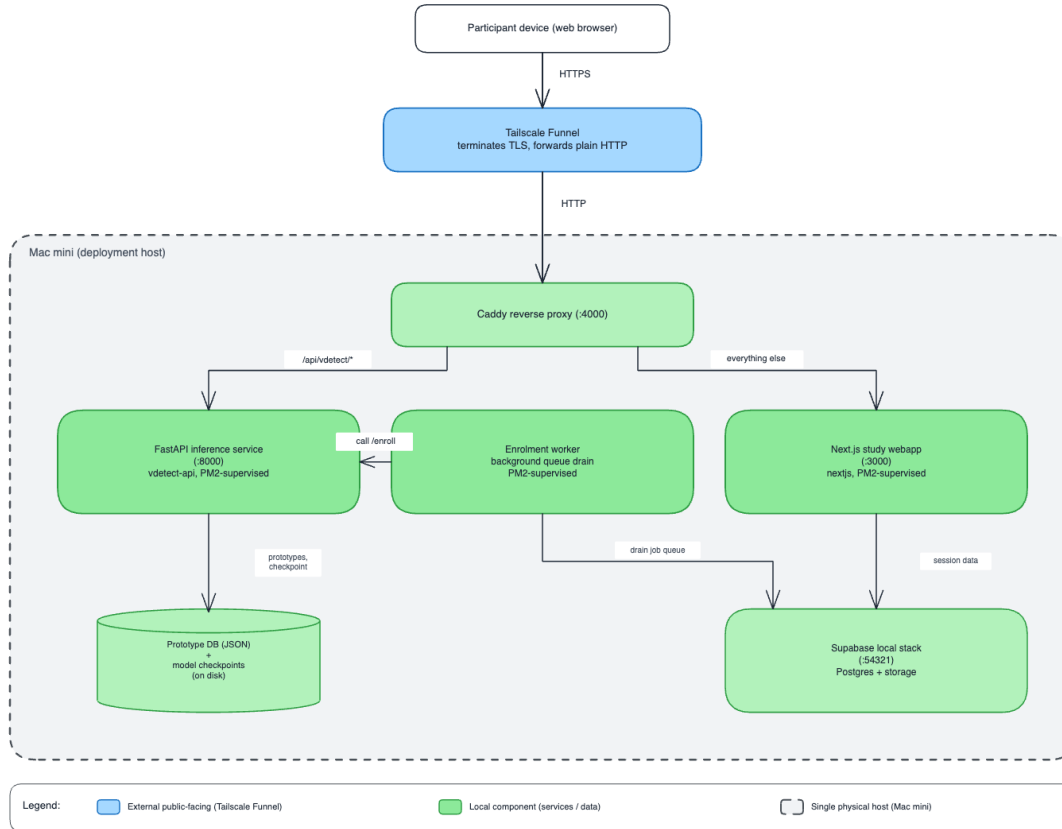


Figure 4 - Diagram showing the project deployment topology

3.5 Study Platform Architecture

Section 3.4 described the inference service that exposes the detector over HTTP. This section describes the participant facing web application that drives the user study, the database schema that backs it, and the two distinct participant journeys it supports.

3.5.1 Architecture Overview

The study platform is implemented as a Next.js 16 application running on a personal Mac mini as described in Section 3.4, served behind the same Caddy reverse proxy and Tailscale Funnel. Two participant facing flows are exposed: `/phase1` which guides the participant through the first phase of the study where they are able to record five voice samples through the web UI, and `/phase2` which guides the participant through the second phase of the study – an anonymous listening task consisting of 20 questions. Each flow consists of a small number of pages (intro, consent, data-collection, and a completion page) and a corresponding set of API routes that control access to the local Supabase stack and to the FastAPI inference service. There is no admin interface which can serve as an additional attack surface containing access to all the aggregated data, all administrative operations such as exporting collected data and purging expired records are command line scripts that are run directly on the host.

3.5.2 Database Schema and Ethics-driven Design

The database schema (shown below) is shaped by the privacy commitments described in Section 3.1.5. Participants are stored in a “participants” table keyed only by a randomly generated UUID, with no name or contact information. Voice recordings, computed prototype embeddings, and generated deepfake clips are linked to the participant via foreign keys, but no personally identifying field appears anywhere in the join chain. The voluntary email opt-in at the end of Phase 1 for the follow up listening test is kept in a completely separate “phase1_emails” table that has no foreign key back to participants and stores only a “bucket_day DATE” field (no exact timestamp), to make it impossible to correlate between an email row and a participant row by looking at the precise times they were created, directly enforcing the privacy commitment P4. A single row “study_lifecycle” table records the study’s closing date, the retention sweep script reads this date rather than computing retention from raw “created_at” values, which makes retention event driven rather than time based and prevents accidental deletion of pre-study development data.



Figure 5 - Study database schema

3.5.3 Phase 1: Enrolment Flow

A participant arriving at /phase1 is first shown an information page with a brief description of the study, its purpose, and key privacy considerations. Upon clicking the start button, the user is shown a consent screen from which the user can access the corresponding participant information sheet and consent forms in PDF format. The points from the consent form are also shown directly on the page, and the user cannot progress until they accept all of the statements and agree to take part in the study by clicking all of the checkboxes individually or using the master “agree all” checkbox at the bottom. After the user grants their consent, the application creates a UUID-keyed row in the participants table and the user is redirected to the recording page which captures five fixed sentence recordings using the MediaRecorder API. The recordings are held entirely in the browser memory until the participant reaches the end of the fifth sentence and clicks finish at which point all five recordings are uploaded sequentially to the private “recordings” bucket into local Supabase Storage and indexed as rows in the “recordings” table.

Once all five uploads finish, the browser enqueues a row in the “enrollment_jobs” queue and the user is redirected to a polling page. The background enrolment worker described in Section 3.4 then claims the job, downloads the recordings, computes the prototype embedding via the “DetectionService.enroll_speaker” Python interface, persists it to the “prototype_embeddings” table, and marks the job as complete. The browser polls the job’s status every two seconds and, upon completion, redirects the participant to the final page which has the optional email opt-in section. The reason all of the recordings are batch uploaded only when the participant clicks finish is so that they can abandon the study at any point before then and essentially withdraw from the study. In this case, since the participant has not clicked finish, we will have no audio data or generated prototype for the participant, and therefore cannot generate any deepfake clips of their voice. The only persistent trace of the visit is the initial “participants” row that was created for them at the consent stage, which carries no audio recordings or personally identifiable information.

3.5.4 Phase 2: Listening Task Flow

A participant arriving at /phase2 is presented with a similar initial screen, followed by an information page with the corresponding consent form and participant information sheet for Phase 2 of the study. Once the user consents to participate in Phase 2, a “phase2_sessions” row is created and a random order is generated for the 20 clips which is stored in the new “phase2_sessions” row. The participant is then redirected to the task page, where they are able to go through and fill out their guesses for each of the 20 audio clips. Each question fetches a short-lived signed URL of the audio sample so the user can hear it, and a “responses” row is submitted for each clip containing the participant’s guess. After the final response is submitted, the participant is taken to the results page which shows them how they fared against the automatic detection system. Once a response has been submitted, it cannot be individually identified or withdrawn since the schema deliberately does not link Phase 2 responses back to Phase 1 participants and Phase 2 by itself does not request any information except the user’s guesses. While this preserves full anonymity, it means that once an answer has been submitted it cannot be withdrawn. A “phase2_clips.active” flag supports soft retirement of clips when the underlying source participant, whose voice was used to generate the audio, withdraws from Phase 1, preserving the integrity of already submitted anonymous responses without keeping the withdrawn participant’s audio in the active pool.

Phase 1 (top) enrolls a participant and computes their prototype; the five recordings are held in browser memory and uploaded as a single batch only when the participant clicks Finish, so abandoning the recording page before that point persists no audio data. Phase 2 (bottom) is a fully anonymous listening task with no schema-level link back to Phase 1 participants.

Phase 1 — Enrolment

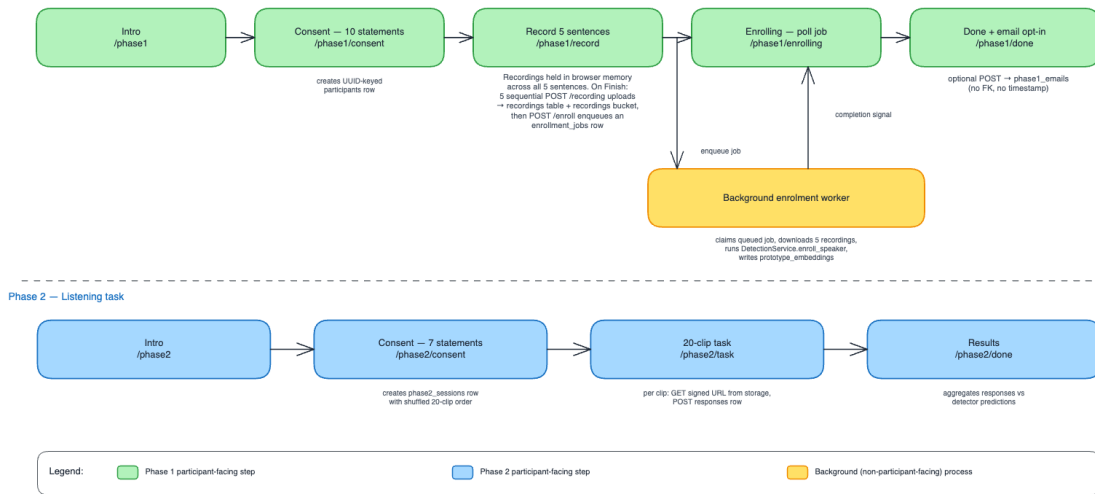


Figure 6 - Diagram illustrating participant flows through the study platform

3.6 User Study

The personalised detector is evaluated through a two phase user study designed to address research question RQ3: whether a personalised deepfake detector can be deployed and evaluated end-to-end through a study with informed participants, and what baseline accuracy those participants achieve on a binary detection task. Because the Phase 2 clip pool is drawn from Phase 1 speakers previously unseen by the detector, it also serves as an additional out of distribution data point bearing on RQ2. The FiLM versus non-FiLM contrast under RQ2 itself is addressed by an offline benchmark evaluation in Chapter 5, not by the user study. The rest of the section sets out the study design, the listening task protocol, and the participant population.

3.6.1 Study Design

Per the recommendation from the QMUL research Ethics Committee, the study is conducted in two separate phases. Phase 1 is the enrolment phase: each participant records five fixed, pre-approved sentences which are used for a number of things. Firstly, they are used to compute the participant's speaker prototype for the FiLM layer. Secondly, they are used to build the pool of bonafide reference clips that are used to create deepfake audio clips and serve as the bonafide clips mixed into the listening test which is Phase 2. In the listening test, a new or returning participant is shown twenty short audio clips drawn from a curated pool, each either bonafide or a deepfake generated using F5-TTS conditioned on a voice from Phase 1. The participant is asked to judge whether they think each clip is real or fake, and provide a confidence rating. Every Phase 2 participant is shown the same pool of 20 audio clips but in a randomly generated order, which enables direct clip comparison between the human judgements and the detector's predictions on the same clips while controlling for any positional or ordering bias.

3.6.2 Listening Task Protocol

Phase 2 is a two-alternative forced choice (2-AFC) task augmented with a confidence rating. For each of the twenty clips in the session, the participant is shown an audio player with no additional visual cues. The participant can replay the clip as many times as they want, and is asked to submit a binary judgement whether they think the clip is a

real or fake voice alongside their perceived confidence rating on a 1-5 scale. There is no time limit on any trial and no feedback is given during the session.

A forced binary judgement with a separate confidence rating was chosen over a single continuous rating scale so that participant decisions remain directly comparable to the detector's binary predictions. The confidence rating still gives the participant a way to express uncertainty which is useful for later analysis in Chapter 5, particularly the contrast between low confidence wrong answers and high confidence wrong answers. At the end of the session, the participant is shown an aggregate score for their twenty answers and a comparison against the detector's predictions on the same clips.

3.6.3 Participants and Clip Pool

After the study was approved by the QMUL Research Ethics Committee, participants were recruited through convenience sampling from the researcher's personal, professional, and academic networks. Inclusion criteria were intentionally minimal: adults aged 18 or over with sufficient English proficiency to read the five prompt sentences aloud and complete the listening task. Eight participants completed Phase 1 enrolment, contributing the bonafide recordings and reference voices that seeded the Phase 2 clip pool. Phase 2 listening was completed by nine participants in total, the majority of whom were also Phase 1 contributors. The small sample size is an acknowledged limitation, its implications for statistical power are addressed in the limitations section of Chapter 5.

The Phase 2 clip pool consisted of twenty short audio clips: ten bonafide recordings drawn directly from Phase 1 and ten deepfakes generated with F5-TTS using the voices from Phase 1. Clip selection was performed manually by the researcher and the final set of 20 audio clips were compiled before the start of Phase 2 to ensure that every Phase 2 participant saw the same set of clips in a randomised order.

Chapter 4: Implementation

4.1 Detector Training Pipeline

The detector is implemented in PyTorch 2.x with the WavLM backbone loaded using Hugging Face Transformers. Both training and inference were run on an Apple M1 Pro using MPS acceleration. The *forward()* method of the *TwoStreamDetector* class realises the architecture from Section 3.2 in roughly ten lines of code. Each call routes the input waveform through both backbones in parallel, concatenates their outputs into a 1696-dimensional fused embedding, applies FiLM modulation when a speaker prototype is supplied, and passes the result through the MLP classification head. The conditional FiLM call is the simplest realisation of the augment rather than replace property from Section 3.3.4: when no prototype is supplied, the block is simply skipped and the call collapses to the unconditioned baseline.

```
def forward(
    self,
    wav: torch.Tensor,
    attention_mask: Optional[torch.Tensor] = None,
    speaker_proto: Optional[torch.Tensor] = None,
) -> Dict[str, torch.Tensor]:
    with torch.set_grad_enabled(self.training):
        wavlm_features = self.wavlm_backbone(wav,
attention_mask=attention_mask).last_hidden_state

        wavlm_emb = self.wavlm_pool(wavlm_features)
        aasist_emb = self.aasist(self._prepare_aasist_input(wav))
        fused = torch.cat([wavlm_emb, aasist_emb], dim=-1)

        if speaker_proto is not None:
            fused = self.film(fused, speaker_proto)

        logit = self.fusion_head(fused).squeeze(-1)
        return {"logit": logit, "score": torch.sigmoid(logit)}
```

Training was driven by binary cross-entropy with logits and the AdamW optimiser, with three parameter groups at different learning rate scales – the fusion head at 1.0x, the WavLM pooling network at 0.5x, and any unfrozen WavLM layers at 0.1x, reflecting how recently each component was initialised.

The MPS backend imposed two specific software constraints: MPS does not support float64 tensors, so the ASVspoof dataset loader was modified to wrap labels in *torch.tensor(label, dtype=torch.float32)*; and *pin_memory* is a CUDA exclusive DataLoader optimisation, so its flag was guarded on *device == "cuda"*. The thermal envelope of the M1 Pro was the practical constraint on training time as each fusion run sustained roughly five hours of continuous MPS load, so training was scheduled to run overnight to allow the computer to run without any additional applications taking up processing power or memory. The computer was also placed on four various small household items to raise it up from the desk and allow enhanced airflow to reach its underside. At one point an improvised cooling system was attempted in the form of an ice pack and a hair dryer blowing cold air under the raised machine, however although the CPU temperature fell by a few degrees, the setup was deemed impractical as it required constant manual supervision and the effect didn't prove worthwhile.

4.2 Personalisation: Enrolment and Prototype Storage

Personalisation is realised in two relatively short pieces of code: the prototype computation in `vdetect/enrollment.py` and the cross process MPS coordination lock in `vdetect/mps_lock.py`.

Once the enrolment worker has extracted fused embeddings for a speaker's 3-5 reference recordings, `compute_prototype` averages them element wise and L2-normalises the result. The function is the only piece of mathematics specific to personalisation, everything else is for storage and retrieval.

```
def compute_prototype(embeddings: torch.Tensor, normalize: bool = True)
-> torch.Tensor:
    proto = embeddings.mean(dim=0)
    if normalize:
        proto = proto / (proto.norm(p=2) + 1e-8)
    return proto
```

Because the enrolment worker, the FastAPI service, and the offline scoring CLIs all load PyTorch models onto the same MPS device, concurrent loads can exhaust the unified memory pool. The `mps_lock` context manager serialises them using an `fcntl.flock(LOCK_EX)` advisory lock on `/tmp/vdetect-mps.lock`. It is acquired only around the model loading phase, once a model is in memory PyTorch's internal scheduler handles concurrent inference safely at study volume and is released automatically when the process exits, so a crashed holder cannot wedge the system.

```
@contextmanager
def mps_lock(owner: str) -> Iterator[None]:
    path = _lock_path()
    print(f"[{owner}] waiting for MPS lock at {path}...", flush=True)
    fd = os.open(path, os.O_RDWR | os.O_CREAT, 0o644)
    try:
        fcntl.flock(fd, fcntl.LOCK_EX)
        print(f"[{owner}] acquired MPS lock", flush=True)
        try:
            yield
        finally:
            fcntl.flock(fd, fcntl.LOCK_UN)
    finally:
        os.close(fd)
```

4.3 Study Web Application

The study web application is implemented as a Next.js 16 App Router project with `shadcn/ui` front-end components, communicating with the local Supabase stack via the service-role API for writes and short lived signed URLs for Phase 2 audio fetches. The most distinctive piece of front-end code is the Phase 1 record page's Finish handler which realises the batch upload functionality after the user clicks finish for privacy.

```
const handleNext = async () => {
    if (!recorder.blob) return;

    // Save this sentence's blob + duration locally
    blobsRef.current[currentIdx] = recorder.blob;
    durationsRef.current[currentIdx] = recorder.durationSec;
```

```

const isLast = currentIdx === TOTAL - 1;

if (!isLast) {
  setCompleted((prev) => {
    const next = prev.slice();
    next[currentIdx] = true;
    return next;
  });
  recorder.reset();
  setCurrentIdx((i) => i + 1);
  return;
}

// Last sentence, finish, batch-upload all 5 recordings, then enroll
recorder.setUploading();
setCompleted((prev) => {
  const next = prev.slice();
  next[currentIdx] = true;
  return next;
});

const allBlobs = blobsRef.current;
const allDurations = durationsRef.current;

try {
  for (let i = 0; i < TOTAL; i++) {
    const blob = allBlobs[i];
    if (!blob) {
      throw new Error(`Missing recording for sentence ${i + 1}`);
    }
    setUploadProgress({ current: i + 1, total: TOTAL });
    await uploadRecording(blob, i + 1, allDurations[i]);
  }
} catch {
  toast.error("Upload failed – please try again");
  setUploadProgress(null);
  // Roll the local "completed" flag off the last sentence so the
  finish button re-enables and the user can retry.
  setCompleted((prev) => {
    const next = prev.slice();
    next[currentIdx] = false;
    return next;
  });
  recorder.setReview();
  return;
}

// All uploaded, enqueue enrollment
try {
  const res = await fetch("/api/phase1/enroll", { method: "POST" });
  if (!res.ok) {
    toast.error("Enrolment failed – please contact the researcher");
    setUploadProgress(null);
    recorder.setReview();
    return;
  }
  const data = (await res.json()) as { jobId: string };
  router.replace(
    `/phase1/enrolling?jobId=${encodeURIComponent(data.jobId)}`,
  );
} catch {
  toast.error("Enrolment failed – please contact the researcher");
  setUploadProgress(null);

```



```
        recorder.setReview();  
    }  
};
```

Phase 2 audio fetching follows a similar privacy conscious pattern, each clip is served from local Supabase Storage as a signed URL with a 20 minute time to live (TTL), proactively refreshed by the client roughly two minutes before expiry so that long loading clips never break mid trial. The screenshots below show the participant facing UI for Phase 1 audio recording and Phase 2 listening respectively.

PHASE 1 — RECORDING

Read each sentence aloud



SENTENCE 1 OF 5

The morning sun cast a warm golden light over the quiet village
as the birds began to sing

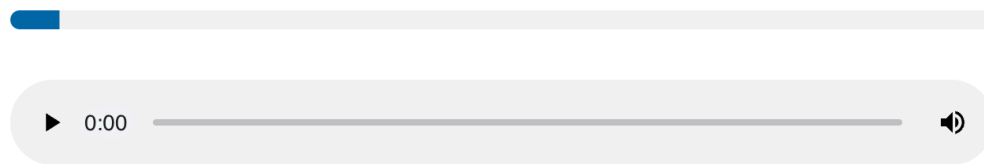
Ready when you are.



Figure 7 - Screenshot of the Phase 1 recording page

PHASE 2 — LISTENING TEST

Clip 1 of 20



Is this clip real or fake?

☐ Real ☐ Fake

How confident are you?

☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5

1 — Not confident

5 — Very confident

Figure 8 - Screenshot of the Phase 2 listening task

4.4 Self-hosted Deployment

The PM2 ecosystem configuration declares three application processes that make up the deployed system, together with their environment variables. Two of these, the Tailscale Funnel URL and the Supabase service-role key are committed to source control as placeholders and substituted at deploy time from *tailscale funnel status* and *supabase status* command outputs. Since *git pull* restores the placeholders on every code update, the substitution has to be repeated each time.

```
env: {
  NODE_ENV: 'production',
  PORT: '3000',
  PATH:
    '/opt/homebrew/opt/node@22/bin:/opt/homebrew/bin:/usr/bin:/bin',
  NEXT_TELEMETRY_DISABLED: '1',
  NEXT_PUBLIC_SITE_URL: '<TAILSCALE_FUNNEL_URL>',
  SUPABASE_URL: 'http://127.0.0.1:54321',
  SUPABASE_SERVICE_ROLE_KEY: '<FROM_SUPABASE_STATUS>',
  VDETECT_BASE_URL: 'http://127.0.0.1:8000',
}
```

To survive a cold reboot in case the computer suddenly lost power or was rebooted by mistake while the study was in progress, the *pm2 startup* command was run to generate a launchd entry in macOS and a small *boot_stack.sh* script was made to make sure the system waits for Docker to launch before bringing up the Supabase DB.

Chapter 5: Evaluation & Results

5.1 Experimental Setup

This chapter draws on three datasets chosen to span the in domain to out of distribution (OOD) spectrum. ASVspoof 2019 Logical Access (Wang et al., 2020) provides the in domain dataset used for training, threshold tuning, and the headline EER result. WaveFake (Frank and Schönherr, 2021) provides cross architecture out of distribution audio drawn from seven distinct synthesis systems. Finally, In-the-Wild (Müller et al., 2026) provides real world out of distribution audio scraped from publicly available recordings of 54 public figures, with bonafide and deepfake audio clips for each speaker.

Dataset	Bonafide	Spoof	Used for
ASVspoof 2019 LA – train	2,580	22,800	Model training
ASVspoof 2019 LA – dev	2,548	22,296	Threshold tuning, early stopping
ASVspoof 2019 LA – eval	7,355	63,882	In domain test
WaveFake	2,700	18,900	4,000 clip balanced OOD sub-sample
In-the-Wild	19,963	11,816	4,000 clip balanced OOD sub-sample

Table 5 - Datasets used for training and evaluation

The primary metric used throughout is Equal Error Rate (EER) which is the operating point on the score vs. threshold curve at which the false acceptance and false rejection rates are equal. Thresholds are computed on the dev split of ASVspoof 2019 LA and applied unchanged to the eval split and to the OOD evaluations to avoid overfitting on out of distribution audio. AUC and accuracy are reported alongside EER throughout to give a fuller picture of the score distribution behaviour, particularly in the OOD setting where detector scores tend to saturate near the top of the $[0, 1]$ range.

The paired FiLM comparison in Section 5.3 uses a bootstrap procedure with 1,000 resamples drawn with replacement at the clip level, sampling pairs of scores (the same clip evaluated with and without the speaker prototype) rather than individual scores. This makes it so that any difference between the FiLM and no FiLM conditions can be attributed to the modulation rather than which clips were randomly drawn. Per speaker 95% confidence intervals on the ΔEER are computed by resampling within each speaker’s paired clip pool, with statistical significance reported at $\alpha = 0.05$.

5.2 In Domain Benchmark – RQ1

The in domain benchmark establishes the detector’s baseline performance on the distribution it was trained on. Three model variants were evaluated on the ASVspoof 2019 LA evaluation split (71,237 samples, of which 7,355 are bonafide and 63,882 are spoof). First, the WavLM baseline, in which the pretrained WavLM base and encoder are

held frozen and only an attentive statistics pooling layer and a small classification head are trained on top. Second, the AASIST encoder, evaluated using the original authors pretrained checkpoint without further fine tuning. Finally, the proposed fusion architecture in which both backbones are held frozen and only the fusion head is trained.

Variant	EER	AUC	Accuracy	F1	Trainable params
WavLM baseline	5.52%	0.9861	89.64%	0.9388	886K
AASIST (pretrained, no fine tuning)	4.85%	0.9907	74.68%	0.8357	0
Fusion (WavLM + AASIST + MLP head)	1.07%	0.9991	92.17%	0.9543	1.03M

Table 6 - In domain results on the ASVspoof 2019 LA dataset

The fusion architecture achieves an EER of 1.07% on the evaluation set, outperforming both individual streams by 3-4 percentage points. The improvement comes at a modest training cost, the fusion head accounts for only 1.03 million trainable parameters, with the 94.6 million parameter backbones held frozen throughout. This directly addresses RQ1 – combining self-supervised speech features with spectro-temporal artefact features under late fusion produces a measurable in domain improvement over either stream alone, and does so without retraining either of the underlying encoders.

For context, the original AASIST paper (Jung et al., 2021) reports 1.13% pooled EER and 0.83% best seed EER on the same evaluation set. The 4.85% figure here is the pretrained checkpoint evaluated off the shelf without additional fine tuning the original authors performed on their training split. This is the appropriate baseline for fusion comparison because both backbones in fusion architecture are likewise frozen at evaluation time. The AASIST checkpoint’s accuracy of 74.68% is lower than WavLM and Fusion despite a better EER because its decision threshold is mis-calibrated relative to the threshold on dev convention used in this work. EER and AUC, which are threshold independent, reflect the underlying ranking quality more reliably than accuracy on this heavily imbalanced benchmark.

This evaluation tests only in distribution performance. The harder test of out of distribution generalisation, and the personalisation comparison that is at the heart of this work, follow in Section 5.3.

5.3 Out of Distribution Evaluation – RQ2

The OOD evaluation tests how the in domain results from Section 5.2 transfer to completely new audio the detector was never trained on. Two benchmarks are used: WaveFake, which provides cross-architecture audio from seven synthesis systems, and In-the-Wild, which provides real world audio of public figures taken from publicly available sources. The In-the-Wild evaluation is the primary test of FiLM personalisation because each speaker contributes both bonafide and deepfake clips, supporting the speaker prototype enrolment described in Section 3.3.

On the 4,000 clip balanced sub-sample of WaveFake, the Fusion detector achieves an EER of 45.48% and an AUC of 0.5463, eliminating the in domain margin almost entirely.

Per architecture EERs span 41.68% (WF7) to 48.87% (WF3), the variation across architectures is small relative to the size of the OOD gap itself. The score distributions are heavily saturated, with mean bonafide and mean spoof scores both above 0.998, leaving little ranking headroom. This demonstrates the OOD generalisation gap discussed in Section 2.5.1, a detector trained exclusively on ASVspoof 2019 LA has effectively no transfer to cross architecture synthetic speech.

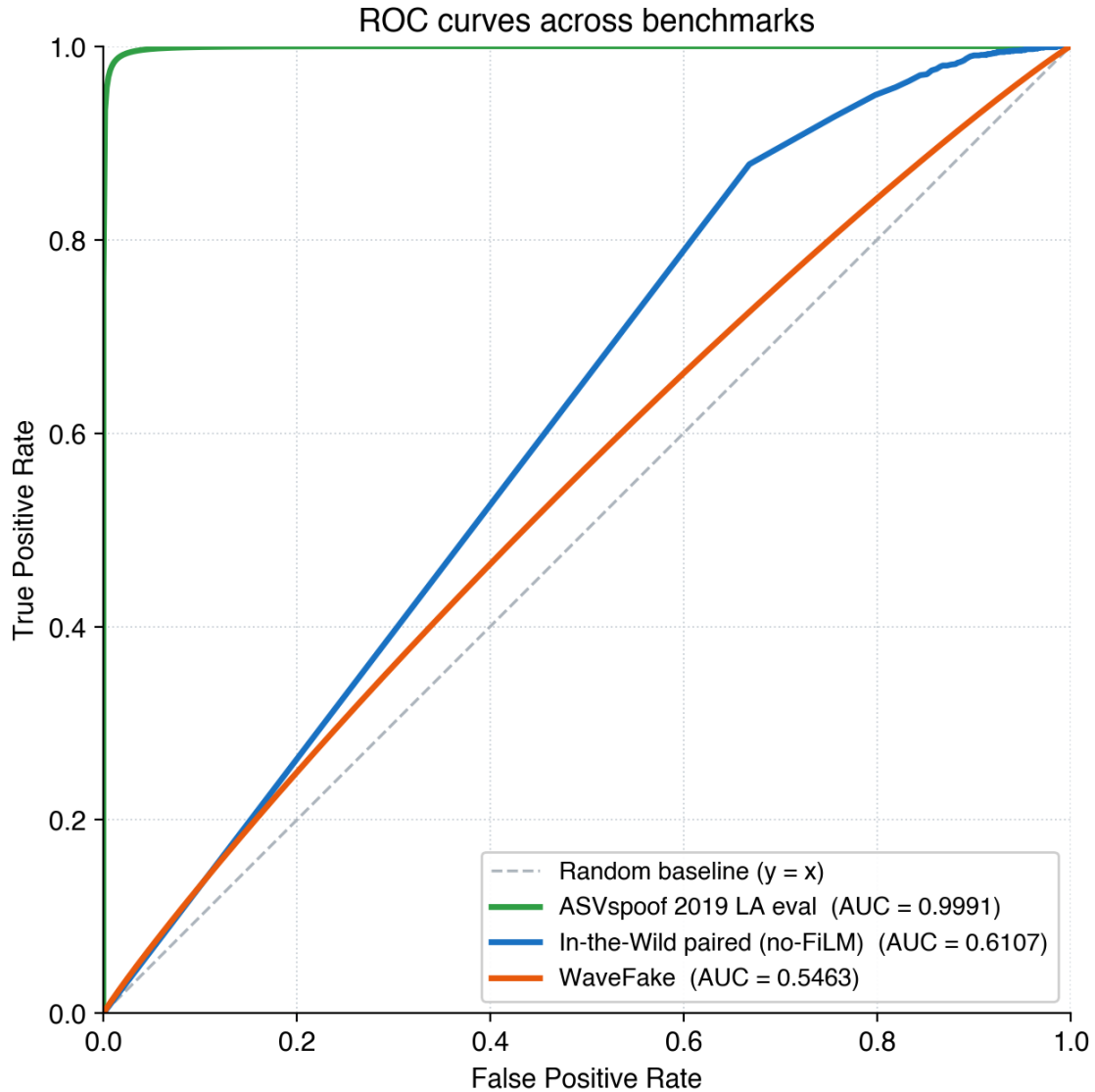


Figure 9 - Graph showing Receiver Operating Characteristic (ROC) curve for the fusion detector across the three evaluation benchmarks

In-the-Wild is evaluated under a paired protocol: each of the 4,000 sampled clips is scored twice in close succession by the deployed vdetect API, once with the speaker's enrolled prototype (FiLM active) and once without (FiLM bypassed), with order randomised per clip. This pairing supports per speaker bootstrap confidence intervals on the Δ EER and eliminates the dominant sampling noise term that confounded an earlier unpaired run in which the apparent improvement was inflated by roughly 35% (Δ EER of -2.03 percentage points unpaired versus -1.30 percentage points paired), only the paired figures will be quoted. The 4,000 clip pool is drawn from 53 speakers since one of the 54 speakers contributed did not have enough samples beyond the three enrolment clips so there was no material to add to the eval pool.

The headline results are summarised in the table below. The Fusion detector with FiLM modulation achieves an EER of 38.20%, compared to 39.50% without modulation – a 1.30 percentage point reduction, with a 95% bootstrap confidence interval of [-2.21 pp, -0.42 pp] over 1,000 samples. Because the entire interval lies below zero, the FiLM improvement is statistically significant at $p < 0.05$. The effect is modest in absolute terms: a one percentage point reduction on a detector that starts at around 40% EER does not close the OOD generalisation gap, it sharpens an already degraded detector.

Mode	EER	AUC	Mean score (real)	Mean score (fake)
No FiLM	39.50%	0.6107	0.9840	0.9993
With FiLM	38.20%	0.6321	0.9799	0.9992
Δ	-1.30 pp	+0.0214	-0.0041	-0.0001

Table 7 - In-the-Wild paired comparison results

The mechanism by which FiLM helps is informative. Fake class scores are already saturated at approximately 1.0 and FiLM barely shifts them (mean $\Delta = +0.0001$). Instead, FiLM lowers mean bonafide class scores by 0.0041 (37 out of 53 speakers shift in this direction), widening the narrow (fake – real) score gap from 0.0153 to 0.0193. AUC improves from 0.611 to 0.632 because the ranking is sharpened in the small region where bonafide and spoof scores overlap. This means FiLM does not learn to flag more spoofs, but rather learns to be slightly more confident that bonafide audio is bonafide.

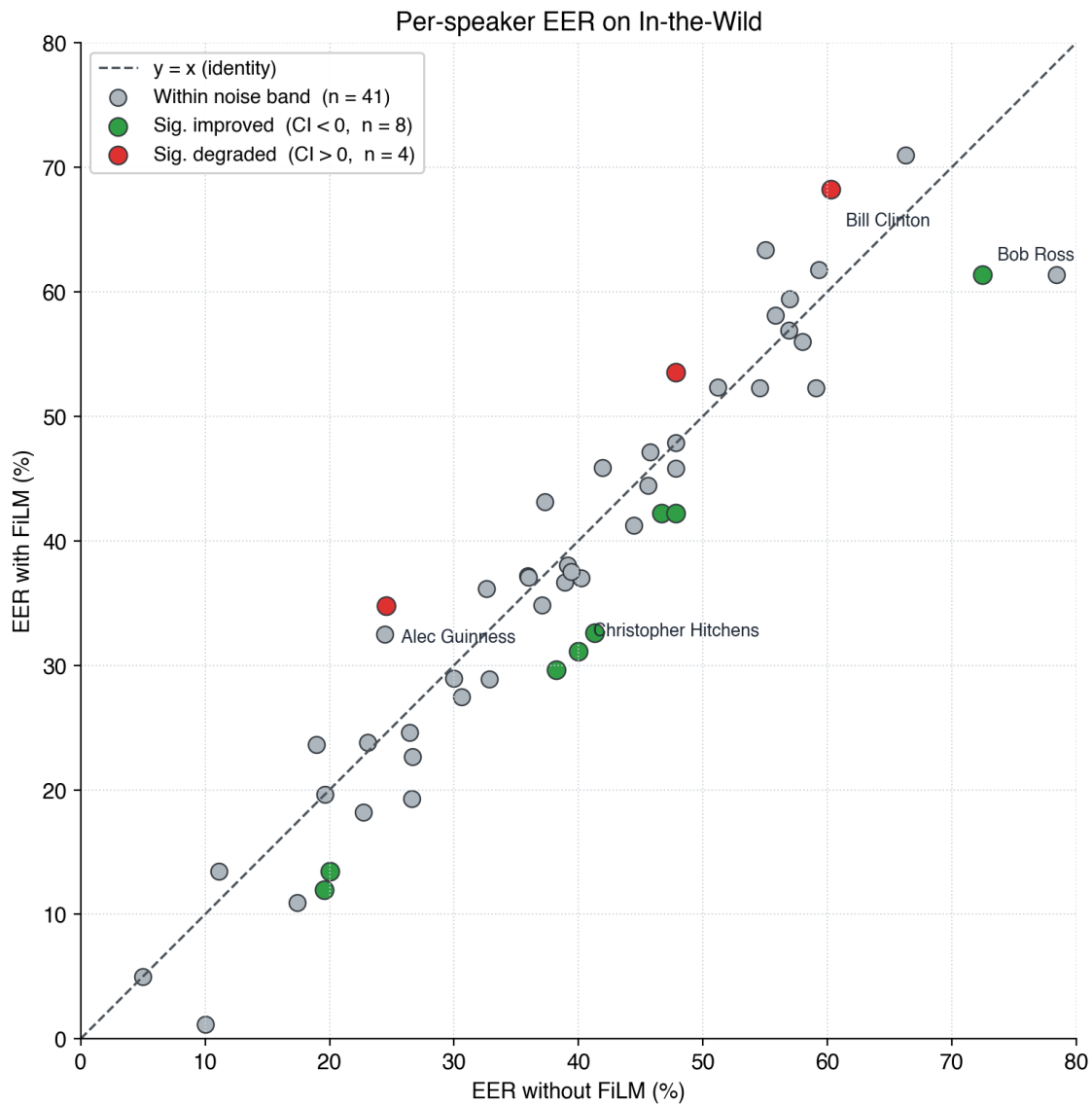


Figure 10 - Scatterplot showing per speaker EER on In-the-Wild

The per speaker breakdown using paired bootstrap CIs reveals genuine heterogeneity. Out of the 53 speakers, 8 (15%) show a statistically significant improvement (95% CI is entirely below zero), 4 speakers (8%) show a statistically significant degradation (95% CI is entirely above zero), and 41 speakers (77%) fall within the noise band. The top improvers include Bob Ross (-11.11 pp), Christopher Hitchens (-8.89 pp), and Donald Trump (-8.54 pp). The top degraders, on the other hand, include Alec Guinness (+10.20 pp), Bill Clinton (+7.95 pp), and Nelson Mandela (+5.68 pp). Although FiLM starts out as the identity function, the trained γ -net and β -net do not stay close to identity on test time prototypes that lie outside the training distribution, and per-speaker outcomes diverge accordingly.

5.4 Listening Study Results – RQ3

The deployed listening study evaluated nine participants on the 20 clip pool described in Section 3.6. Each participant judged each clip as either Real or Fake with a 1-5 confidence rating, with clip order randomised per session. The detector was evaluated on the same 20 clips for direct per clip comparison.

5.4.1 Aggregate Performance

Across the 9 participants and 180 total responses, mean accuracy was 14.11 out of 20 (70.6%) with median 14, range 9-18, and $\sigma = 2.86$. Mean per clip confidence was 4.09 out of 5 and median session duration was 3 minutes 19 seconds. Per session results are reported in the table below.

#	Session	Correct	Real	Fake	Avg conf	Duration
1	f34a4ddc	13/20	6/10	7/20	3.90	2:55
2	76b14192	18/20	8/10	10/10	4.05	4:56
3	2dd64f61	15/20	6/10	9/10	4.25	3:16
4	2a56619c	9/20	4/10	5/10	4.45	3:18
5	c2c29cb61	14/20	7/10	7/10	3.95	4:03
6	36c2353e	18/20	9/10	9/10	4.15	4:20
7	fb83470e	13/20	7/10	6/10	3.70	3:19
8	43337c30	15/20	7/10	8/10	4.35	3:04
9	e97db3cf	12/20	6/10	6/10	4.05	3:36

Table 8 - Per session results for the nine Phase 2 participants

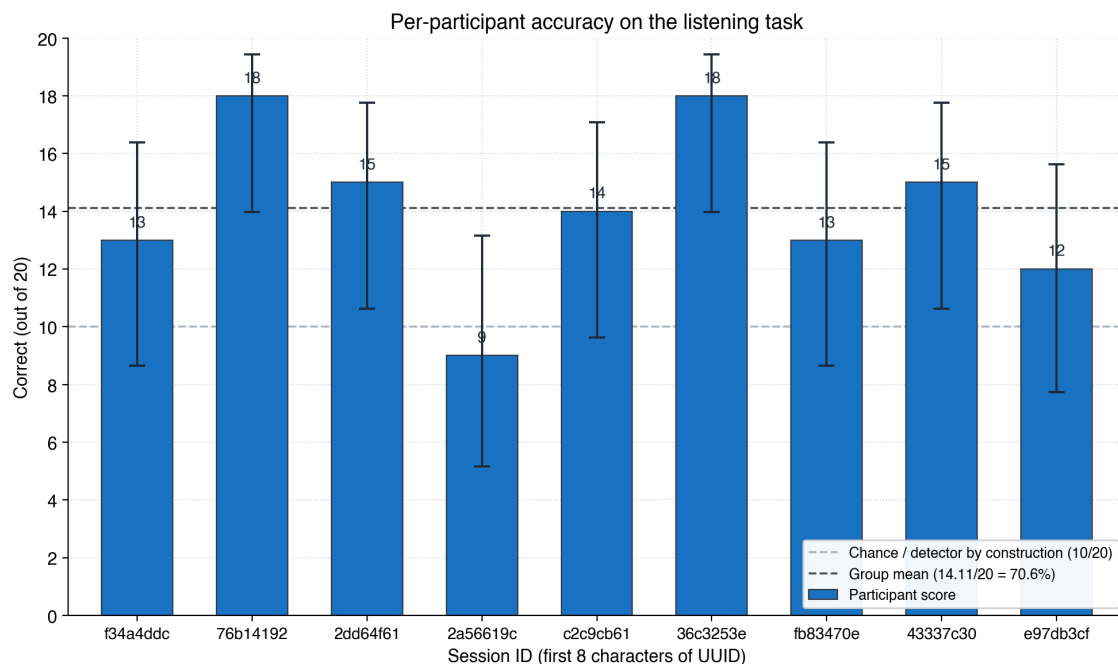


Figure 11 - Per participant accuracy on the 20 clip listening task

The per session scores span a wide range. The two top scorers reached 18 out of 20, the lowest scorer achieved 9 out of 20, a result below chance on real and slightly above on fake clips, while reporting the highest mean confidence of any participant (4.45). With $n=9$ this “confidently wrong” pattern is a single data point, but it suggests that self-reported confidence is not a reliable signal of accuracy at an individual level.

5.4.2 Real / Fake Asymmetry

Participants were correct on 60 out of 90 real clips (66.7%) and 67 of 90 fake clips (74.4%), with mean confidence at 4.17 for real clips and 4.02 for fake ones. The answer distribution was 97 “said fake” versus 83 “said real” across 180 trials, indicating a mild false positive bias showing that participants were slightly more inclined toward labelling clips as fake when uncertain, which helped them on the fake half but hurt them on the real half. This is consistent with the priming effect of being told that the listening test concerns deepfake detection.

5.4.3 Per sentence variation

Sentence 3 (“Please remember to bring your jacket with you today...”) scored only 33% (6/18) when participants were judging bonafide audio, well below the cross sentence average. With only two distinct bonafide clips available from sentence 3, this most likely reflects properties of those specific recordings rather than a sentence level effect. At the opposite extreme, sentence 1 fake clips scored 89% (16/18), suggesting F5-TTS produced particularly identifiable artefacts on this sentence.

5.4.4 Confidence Calibration

The table below breaks down accuracy by self-reported confidence. The pattern is roughly monotonic from confidence 3 to 5, but flat at the top, with confidence 4 (74.3%) marginally outperforming confidence 5 (72.6%). Confidence ratings 1 and 2 were essentially unused.

Confidence	n	Correct	%
1	0	-	-
2	1	1	100% (singleton)
3	43	26	60.5%
4	74	55	74.3%
5	62	45	72.6%

Table 9 - Confidence calibration

5.4.5 Detector Saturation

The personalised Fusion + FiLM detector was evaluated on the same 20 clip pool the participants saw. On every clip in the pool, both bonafide and synthetic, the detector returned a score of approximately 1.0. The per clip AUC was between 0.50 and 0.56 across the model configurations tried. At any sensible threshold the detector classifies every clip as spoof, scoring 10 out of 20 due to the nature of the clip pool – scoring ten correct on the fake clips, and zero correct on the real clips. The saturation is consistent with the OOD generalisation failure documented earlier, the Phase 2 clip pool, comprising browser captured bonafide audio and F5-TTS generated synthetic audio, is

an even further out of distribution shift than In-the-Wild for a detector trained solely on ASVspoof 2019 LA.

5.4.6 Conclusions Drawn for RQ3

Two conclusions can be drawn for RQ3. First, the deployment infrastructure described in Chapters 3 and 4 is operationally complete. The personalised detector was successfully deployed and evaluated end to end through a study with nine informed participants completing 180 trials in total. Second, the human baseline accuracy on the binary detection task is established at 70.6%, with per participant scores ranging from 9 to 18 out of 20. The head to head comparison between human judgement and the deployed detector on the same clip pool, which the study was originally designed to support, is bounded by the score saturation behaviour reported in Section 5.4.5: the detector returns scores near 1.0 for every Phase 2 clip regardless of whether it is bonafide or synthetic, leaving no useful ranking signal at any threshold. What this saturation implies for future deployment of personalised detectors is explored further in Chapter 6.

Chapter 6: Discussion & Conclusion

Chapter 5 establishes three main findings:

1. Fusion of WavLM and AASIST produces a strong in domain detector at 1.07% EER
2. FiLM personalization produces a statistically significant, but modest, EER reduction on out of distribution audio with heterogeneous per speaker effects
3. The personalised detector can be deployed and evaluated end to end with informed participants reaching 70.6% mean accuracy on the listening task

Each finding is bound by limitations that shape how it should be interpreted and what follows from it.

6.1 Limitations

6.1.1 Voice Familiarity Bias in the Participant Pool

Most participants of Phase 2 of the study personally knew several of the Phase 1 source speakers, and therefore had access to a learned mental model of those speakers' actual voices when judging clips. This is precisely the kind of "outsourced familiarity" that the detector itself was designed to provide. The 70.6% human baseline is consequently likely to be inflated relative to a clean general population baseline by an unknown degree, therefore the figure should be considered more as an upper bound on unaided human performance rather than as a population estimate.

6.1.2 Single Training Dataset on Frozen Backbones

The detector was trained exclusively on the ASVspoof 2019 LA dataset with both backbones held frozen. This was a pragmatic choice given the hardware constraints described in Section 4.1, but one that imposes a hard ceiling on how well the model can specialise. Training on a union of public spoofing datasets covering broader synthesis and bonafide distributions, combined with selective unfreezing of the WavLM and AASIST backbones, is the single largest potential performance gain that was missed out on.

6.1.3 FiLM Was Not Trained on a Properly Representative Dataset

The episodic FiLM tuning run once again used ASVspoof 2019 LA, which has a narrow speaker pool and synthesis distribution. FiLM ideally requires a corpus where the same speaker appears as both bonafide and fake, drawn from a diverse synthesis distribution – a structure that In-the-Wild has exactly, however it was instead chosen to be kept purely as an evaluation benchmark.

6.1.4 Out of Distribution Score Saturation

Both In-the-Wild paired evaluation and the Phase 2 clip pool exhibit score saturation, with the detector returning scores near 1.0 regardless of class. The personalisation pipeline does not address this; FiLM marginally widens the bonafide/spoof gap on In-the-Wild but cannot reopen it on the further OOD audio clips used in Phase 2 of the study. Without a score calibration or normalisation strategy, deployed detectors of this kind will most likely continue to fail in this way.

6.1.5 No Personalisation Element in the User Study

The study collects human judgements against the FiLM personalised detector, but does not put participants through an analogue of the enrolment and personalised detection pipeline. There were a couple of approaches that could have been implemented to address this. The study could have leaned into the personalised detection aspect by focusing on a small group of people who all know each other well, such as a small team in a business or friend group with around 10-15 people. Alternatively the study could have gone for a large scale recruitment of participants that may or may not know each other, such as a university cohort and have each participant briefly familiarise themselves with a set of voices, after which point they would be asked to classify a voice sample as fake/real and identify the owner of the voice from the set of voices they familiarised themselves with at the start of the study. The current version of the user study had one large set of people who knew each other's voices well, but there were also a few people out of the main group who did not know anyone in the "big group" and none of the people from the "big group" knew them, therefore muddling up the results if the study chose to focus on the "big group" only.

6.2 Future Work

6.2.1 Redesigned User Study

A scaled up version of the listening study could address both the voice familiarity bias and the absence of a personalisation element in the current design. Stratified recruitment of participants without prior knowledge of the source speakers, such as from a wider university cohort rather than the researcher's direct personal network, would replace the likely inflated upper bound figure with a more grounded general population estimate. The protocol could additionally include a speaker familiarisation step followed by the current questionnaire with an added speaker identification judgement, giving a direct human analogue of FiLM personalisation that the present study lacks.

6.2.2 Training on a Union of Datasets with Selective Unfreezing

A future study could combine a wider range of datasets, such as ASVspoof 2019 LA with WaveFake, MLAAD, and ASVspoof 2024 to span a much broader range of synthesis architectures, languages and recording conditions. MLAAD alone provides bonafide and synthetic audio across 38 languages, addressing the entirely English only training distribution used in this work. Selective unfreezing of the last few WavLM transformer layers, or LoRA adapters applied to the AASIST graph attention layers, would let the model adapt to the broader distribution. This seems to be one of the most direct ways to address the generalisation gap documented in Section 5.3.

6.2.3 Speaker Paired FiLM Training Dataset

Curating or assembling a separate training dataset with speaker-paired bonafide and synthetic clips, drawn from a diverse group of synthesis architectures, would let FiLM see at training time the variation pattern it is currently asked to extrapolate to at inference time. The dataset would need careful construction to keep at least one cross architecture benchmark such as In-the-Wild untouched for evaluation, avoiding any contamination of test data with train data.

6.2.4 Streaming and Real-Time Inference

The current detector operates on whole audio clips. A future system could process audio in streaming fashion, windowing the call audio into overlapping segments, scoring each segment, and accumulating evidence over the duration of the call to support a confidence weighted real time alert if a deepfake voice is detected.

6.2.5 Systematic Exploration of Alternative Two-Stream Pairings

A controlled comparison of two-stream pairings, varying both the speech encoder backbone (Whisper, HuBERT, wav2vec 2.0, etc.) and the spoofing specific stream (AASIST, RawNet2, Res2Net) would establish whether the in domain advantage of the current fusion is architecture specific or general to late fusion designs. Evaluated under the same FiLM personalisation harness, this would also test whether the per speaker heterogeneity seen in Section 5.3 is intrinsic to FiLM or specific to the WavLM + AASIST pairing.

6.3 Overall Conclusion

Personalised audio deepfake detection sits at an awkward intersection – the in domain performance of modern detectors is excellent and continues to improve, but their out of distribution behaviour remains brittle in ways that personalisation at inference time cannot fully address. This dissertation contributes a fusion architecture that reaches 1.07% EER on ASVspoof 2019 LA, a paired bootstrap characterisation of FiLM based personalisation that quantifies its modest, statistically significant, and heterogeneous effect on out of distribution audio, and a deployable end to end system that supports continued empirical work.

This project has been an invaluable learning experience and a significant academic accomplishment for me. I have acquired a multitude of new skills that I look forward to putting to use in my professional career and personal side projects, as well as potential further research in the audio deepfake detection space.

Chapter 7: References

Asuai, C., Arinomor, A., Atumah, C., Kowhoro, I. and Ogheneochuko, D. (2025). Hybrid CNN-LSTM Architectures for Deepfake Audio Detection Using Mel Frequency Cepstral Coefficients and Spectrogram Analysis. *American Journal of Mathematical and Computer Modelling*, 10(3), pp.98–109.

doi:<https://doi.org/10.11648/j.ajmcm.20251003.12>.

Chen, S., Wang, C., Chen, Z., Wu, Y., Liu, S., Chen, Z., Li, J., Kanda, N., Yoshioka, T., Xiao, X., Wu, J., Zhou, L., Ren, S., Qian, Y., Qian, Y., Wu, J., Zeng, M., Yu, X. and Wei, F. (2022). WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6), pp.1505–1518. doi:<https://doi.org/10.1109/jstsp.2022.3188113>.

Chen, Y., Niu, Z., Ma, Z., Deng, K., Wang, C., Zhao, J., Yu, K. and Chen, X. (2025). *F5-TTS: A fairytaler that fakes fluent and faithful speech with flow matching*. [online] Arxiv. Available at: <https://arxiv.org/abs/2410.06885>.

clovaai (2025). *GitHub - clovaai/aasist: Official PyTorch implementation of ‘AASIST: Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks’*. [online] GitHub. Available at: <https://github.com/clovaai/aasist?tab=readme-ov-file> [Accessed 11 Dec. 2025].

ElevenLabs (2025). *ElevenLabs - Generative AI Text to Speech & Voice Cloning*. [online] elevenlabs.io. Available at: <https://elevenlabs.io/about>.

Frank, J. and Schönherr, L. (2021). *WaveFake: A data set to facilitate audio deepfake detection*. [online] Arxiv. Available at: <https://arxiv.org/abs/2111.02813>.

Hassan, K. (2025). *AI Cyber Attack Statistics 2025, Trends, Costs, Defense*. [online] DeepStrike. Available at: <https://deepstrike.io/blog/ai-cyber-attack-statistics-2025>.

Iribe, B., Kumar, A. and Sesame Team (2025). *Crossing the uncanny valley of conversational voice*. [online] Sesame. Available at: https://www.sesame.com/research/crossing_the_uncanny_valley_of_voice.

Jung, J., Heo, H.-S., Tak, H., Shim, H., Chung, J.S., Lee, B.-J., Yu, H.-J. and Evans, N. (2021). *AASIST: Audio anti-spoofing using integrated spectro-temporal graph attention networks*. [online] Arxiv. Available at: <https://arxiv.org/abs/2110.01200>.

Kulkarni, A., Tran, H.M., Kulkarni, A., Dowerah, S., Lolive, D. and Doss, M.M. (2024). Exploring generalization to unseen audio data for spoofing: insights from SSL models. *The Automatic Speaker Verification Spoofing Countermeasures Workshop (ASVspoof 2024)*, pp.86–93. doi:<https://doi.org/10.21437/asvspoof.2024-13>.

Li, X., Chen, P.-Y. and Wei, W. (2025). Where are We in Audio Deepfake Detection? A Systematic Analysis over Generative and Detection Models. *ACM Transactions on Internet Technology*, 25(3), pp.1–19. doi:<https://doi.org/10.1145/3736765>.

Loo, L. (2020). *Understanding common features used : [Audio/Speech Processing Part 1]*. [online] Medium. Available at: <https://leonardloo.medium.com/part-1-audio-speech-processing-understanding-common-features-used-82841d02cc93> [Accessed 11 Dec. 2025].

Luo, Q. and Sivasundari, K. (n.d.). *Whisper+AASIST for DeepFake Audio Detection*.

Müller, N.M., Pavel Czempin, Dieckmann, F., Froghyar, A. and Konstantin Böttinger (2026). *Does audio deepfake detection generalize?* [online] Arxiv. Available at: <https://arxiv.org/abs/2203.16263>.

Okabe, K., Koshinaka, T. and Shinoda, K. (2018). Attentive statistics pooling for deep speaker embedding. [online] ISCA, pp.2252–2256. doi:<https://doi.org/10.21437/interspeech.2018-993>.

Perez, E., Strub, F., Harm de Vries, Dumoulin, V. and Courville, A. (2017). *FiLM: Visual reasoning with a general conditioning layer*. [online] Arxiv. Available at: <https://arxiv.org/abs/1709.07871>.

Priya N V, Pavan H, Prajwal S, Varun R and Vinay A (2025). Deepfake Audio Detection using MFCC Features. *International Journal For Multidisciplinary Research*, 7(1). doi:<https://doi.org/10.36948/ijfmr.2025.v07i01.36558>.

Resemble AI (2025). *Chatterbox-tts*. [online] Available at: <https://github.com/resemble-ai/chatterbox>.

Thawani, G., Shukla, S., Wable, R., Chugh, U. and Gondal, A. (2025). A Novel Approach for Deepfake Audio Detection. *International Research Journal of Modernization in Engineering Technology & Science*, 7(4). doi:<https://doi.org/10.56726/irjmets74467>.

Wang, X., Yamagishi, J., Todisco, M., Delgado, H., Nautsch, A., Evans, N., Md Sahidullah, Ville Vestman, Kinnunen, T., Lee, K.A., Juvela, L., Paavo Alku, Peng, Y.-H., Hwang, H.-T., Tsao, Y., Wang, H.-M., Sebastien Le Maguer, Becker, M., Henderson, F. and Clark, R. (2020). *ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech*. [online] Arxiv. Available at: <https://arxiv.org/abs/1911.01601>.

Xie, Y., Cheng, H., Wang, Y. and Ye, L. (2023). Learning A Self-Supervised Domain-Invariant Feature Representation for Generalized Audio Deepfake Detection. *INTERSPEECH 2023*, pp.2808–2812. doi:<https://doi.org/10.21437/interspeech.2023-1383>.

Chapter 8: Appendix

8.1 Risk Register

ID	Risk	Likelihood	Impact	Actions
1	Computational resource limitations	Low – Apple Silicon M1 Pro has so far proven capable of training the fusion model. Training takes 5-7 hours per experiment	Medium – Could slow down experimentation and iteration cycles	Allow extra time in case training takes longer than expected. Consider using cloud GPU providers for intensive tasks if needed
2	Evaluation dataset access or corruption issues	Low – ASVspoof 2019 LA is already downloaded and has been used, other datasets are publicly available	High – Without having access to this data, not benchmarks can be run, and no comparisons can be made to existing solutions	Maintain backups of datasets on external hard drives, verify dataset integrity, document online dataset locations for easy access
3	Hardware failure or data loss	Low – MacBook is generally quite a reliable computer, but accidental damage could be incurred through drops, spills, etc.	High – Potential loss of code, trained models, or experimental results would take significant time to recover and remake	Use git for version control and store model checkpoints in separate cloud backups
4	Human participants not found for customised voice testing	Low – Friends, family, and co-workers are available for testing	High – Would make it impossible to evaluate personalised protection and community protection	Discuss participation in advance and explain that there is no sensitive information at stake except voice data
5	Multi-shot personalisation performance does not meet expectations	Medium – This is a complex feature, and the final implementation is not guaranteed to yield improved generalisation capabilities	High – Multi-shot personalisation is a key differentiating factor of this project, and poor results would make the	Start implementation early to have time to iterate and explore possible other implementations of personalised protection, frame research as

			contribution less significant	exploratory if results are negative
--	--	--	-------------------------------	-------------------------------------

Table 9 - Risk register

8.2 Study Ethics Approval



School of Electronic Engineering
& Computer Science
Peter Landin Building, 10 Godward Square
London E1 4FZ

Devolved School Research Ethics Committee
Email: eeecs-ethics@qmul.ac.uk

30 March 2026

Miss Soumya Vanka
School of Electronic Engineering and Computer Sciences
Queen Mary University of London

Cc: Egor Vert

Dear Soumya Vanka

RE: DSEEC26.027 - Personalised Audio Deepfake Detection: A Human Enrolment and Evaluation Study.

Having received a Research Ethics Application for the study detailed above, the proposed work has been deemed low-risk and will not require the scrutiny of the full Queen Mary Research Ethics Committee (QM REC). It has been reviewed by the Electronic Engineering and Computer Science Devolved School Research Ethics Committee (EECS DSREC) and I am pleased to inform you that, based on the information provided, this application has now been **approved**.

- **Date of Approval: 30 March 2026**
- **Approval End Date: 29 March 2029**

The application was approved subject with the following **advice**:

1. For Phase-1, to set a clear specific date for data storage and add this information on the Participant documents.
2. For Phase 2, to add on the Participant Information sheet the information (description) about the platforms the participants will use.

Please consider the above points carefully before you start your research.

Extensions and Amendments:

If you propose to make an [amendment to the research after approval](#) or extend the study beyond the date of the approval expiry date given above, then the EECS DSREC should be contacted (eeecs-ethics@qmul.ac.uk).

QM REC Approval letter – Low-risk studies
V5.0

Figure 12 - Ethics study approval

8.3 GenAI Tools – Accountability Statement

ECS635U – Project Module

Generative AI Tools – Student Accountability Statement

The completed and signed statement must be included in the appendix of the progress and final reports.

Student Name: **Egor Vert**

Project Title: **Personalised Audio Deepfake Detection**

Supervisor Name: **Soumya Sai Vanka**

1. Have you used Generative AI tools for your project?

YES

If yes, please complete sections 2 and 3 and sign the declaration below.

If no, please sign the declaration below.

2. Please provide details of all the AI tools that you used in your project work.

AI Tool	Version	Publisher	Access URL
Cursor (tab autocomplete)	n/a	Cursor	https://cursor.com/

3. If you have used Artificial Intelligence (AI) tools in any part of your project please include in table format how, where, and why these tools were used. This should include references to the relevant parts of your project work — for example:

- Specific chapters or sections of the report (e.g. literature review, methodology, requirements)
- Software development (e.g. AI-assisted code generation, debugging, documentation)
- Interview or survey materials (e.g. AI-generated or refined questions/prompts)
- Datasets (e.g. AI-assisted data cleaning, labelling, or synthesis)
- The showcase video (e.g. AI-assisted scripting, narration, or editing support)

Be specific and transparent about your use of AI tools, including both direct outputs (e.g. text, code, images) and indirect uses (e.g. idea generation, editing assistance). The goal is to ensure academic integrity and provide a clear record of how AI contributed to your work.

Project Section	Description	Justification
Software development	Used default tab autocomplete inside my code editor	Used tab to accept only short code snippets I understand and would have written myself to save time and carefully reviewed it after accepting the suggestion

3. Generative AI Student Declaration

- I have declared all uses of AI tools (e.g., coding assistants, text generators, data analysis tools) in this project.
- I have reviewed and verified all AI-assisted content to ensure accuracy and suitability.
- I take full responsibility for the originality and integrity of my project submission.
- I have referenced all sources, including those suggested or produced with AI support.
- I have not used AI in ways that undermine academic honesty (e.g., plagiarism, unverified code submission).
- My use of AI was supplementary and did not replace my own independent learning and problem-solving.

Signed: Egor Vert Date: 12th May 2026